学校代码 10530　　学　号 202331510117

分 类 号 O241.82　　密　级 公　开

湘潭大学

博 士 学 位 论 文

基于高阶有限元的变密度拓扑优化数值方法设计及其实现与应用

学 位 申 请 人 何亮

指 导 老 师 魏华祎 教授

学 院 名 称 数学与计算科学学院

学 科 专 业 数学

研 究 方 向 微分方程数值方法及应用

二〇二六年四月十日

基于高阶有限元的变密度拓扑优化数值方法设计及其实现与应用

学 位 申 请 人 何亮

导师姓名及职称 魏华祎 教授

学 院 名 称 数学与计算科学学院

学 科 专 业 数学

研 究 方 向 微分方程数值方法及应用

学位申请级别 理学博士

学位授予单位 湘潭大学

论文提交日期 二〇二六年四月十日

Design, Implementation and Applications of Numerical Methods for Density-Based Topology Optimization Based on High-Order Finite Elements

Candidate Liang He

Supervisor Professor Huayi Wei

College School of Mathematics and Computational Sciences

Program Mathematics

Specialization Numerical Methods and Applications for Differential Equations

Degree Doctor of Science

University Xiangtan University

Date April 10, 2026

湘潭大学
学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：　　日期：　　年　　月　　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权湘潭大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

涉密论文按学校规定处理。

作者签名：　　日期：　　年　　月　　日

导师签名：　　日期：　　年　　月　　日

摘　要

连续体结构拓扑优化是复杂工程结构轻量化与高性能设计的重要数学方法。尽管基于变密度方法的连续体结构拓扑优化已形成较为系统的理论与算法体系，但在高精度分析、复杂局部约束建模与求解以及高效计算实现方面仍面临关键数值挑战。其中，传统低阶位移有限元在局部应力场刻画、近不可压缩材料分析及局部应力约束评估等方面存在局限，而先进数值方法与高性能计算平台之间亦缺乏统一实现框架。针对上述问题，本文围绕连续体结构拓扑优化中的关键数值问题，系统研究高精度离散方法、复杂局部约束优化建模及其高性能实现。

针对高精度离散建模问题，本文系统比较了任意次拉格朗日有限元在变密度拓扑优化中的离散表现及数值特征。通过考察不同单元类型、不同离散阶次以及单元密度与节点密度两类设计变量表征，分析了它们对优化构型、分析精度与计算性能的影响，揭示了高阶拉格朗日有限元在典型变密度拓扑优化问题中的适用特征与精度优势。在此基础上，针对传统单分辨率框架中高精度物理分析与高分辨率拓扑描述难以兼顾的问题，构建了基于高阶有限元的多分辨率拓扑优化计算框架。该框架通过设计网格与分析网格的解耦，建立了设计变量到物理密度的映射机制，以及相应的优化列式、刚度矩阵数值积分与灵敏度分析方法，并进一步推广到应力约束拓扑优化问题。研究结果表明，该框架能够在控制计算规模的同时兼顾结构分析精度与拓扑表达分辨率，并在应力约束拓扑优化问题中表现出良好的适用性。

针对复杂局部约束与特殊材料情形下的拓扑优化建模问题，本文进一步研究了基于任意次胡张混合有限元的高精度拓扑优化方法。基于 Hellinger–Reissner 混合变分原理，本文系统建立了线弹性问题的应力–位移混合描述、任意次胡张混合有限元离散框架及其相应的拓扑优化列式，并通过低阶稳定化处理与复杂边界条件下顶点应力连续性的部分松弛策略，提升了该方法在非结构网格、低阶情形及复杂工程边界下的适用性，进一步面向最小柔顺度、近不可压缩材料与局部应力约束等典型问题构建了相应的数值求解框架。研究结果表明，任意次胡张混合有限元方法在缓解体积闭锁以及提升局部应力约束评估的可靠性方面具有明显优势。

面向前述高精度离散模型与优化算法的统一实现需求，本文依托 FEALPy 开源智能 CAX 计算引擎，设计并实现了高性能拓扑优化软件平台 SOPTX。该平台在继承 FEALPy 底层张量抽象与多后端机制的基础上，围绕拓扑优化闭环构建了面向领域问题的分层架构与核心模块体系，实现了分析过程、灵敏度分析与优化更新之间的模块化解耦。在关键技术层面，平台重点发展了基于解析预计算与张量收缩的高效矩阵组装与算子缓存机制、自动微分驱动的灵敏度分析实现，以及面向 NumPy、PyTorch、JAX 等不同后端与 CPU/GPU 异构设备的性能可移植执行机制。通过典

型数值实验，进一步验证了 SOPTX 在高效矩阵组装与自动微分灵敏度分析方面的正确性与计算效率，以及在多后端与异构设备切换方面的扩展性与计算性能。该平台为本文所提出的多类拓扑优化模型与数值方法提供了统一、可复现且可扩展的软件实现支撑。

综上，本文围绕连续体结构拓扑优化中的若干关键数值问题，从高阶有限元离散方法、多分辨率高精度计算、任意次胡张混合有限元建模到高性能软件平台实现，形成了一条贯通“高精度离散建模–复杂局部约束优化求解–统一软件实现验证”的系统研究主线。相关研究从数学模型、离散格式、灵敏度分析到软件实现与数值验证，建立了较为完整的理论–方法–实现研究链条，为连续体结构拓扑优化中高精度有限元离散、复杂局部约束建模及工程可复现计算提供了较为系统的理论、方法与平台支撑。

关键词: 高阶有限元；胡张混合有限元；多分辨率拓扑优化；应力约束拓扑优化；拓扑优化软件平台 SOPTX

Abstract

Continuum structural topology optimization is an important mathematical approach for the lightweight and high-performance design of complex engineering structures. Although density-based continuum structural topology optimization has developed into a relatively systematic theoretical and algorithmic framework, it still faces several critical numerical challenges in high-accuracy analysis, the modeling and solution of complex local constraints, and efficient computational implementation. In particular, conventional low-order displacement-based finite elements exhibit clear limitations in the characterization of local stress fields, the analysis of nearly incompressible materials, and the evaluation of local stress constraints. Meanwhile, a unified implementation framework that can systematically bridge advanced numerical methods and high-performance computing platforms is still lacking. To address these issues, this dissertation systematically investigates high-accuracy discretization methods, optimization modeling under complex local constraints, and their high-performance implementation for several key numerical problems in continuum structural topology optimization.

To address high-accuracy discretization modeling in topology optimization, this dissertation systematically compares the discrete behavior and numerical characteristics of arbitrary-order Lagrange finite elements in density-based topology optimization. By examining different element families, different polynomial orders, and two design-variable representations, namely element-based and node-based density representations, the effects of these factors on optimized topologies, analysis accuracy, and computational efficiency are analyzed, thereby revealing the applicability features and accuracy advantages of high-order Lagrange finite elements in typical density-based topology optimization problems. On this basis, to overcome the difficulty of simultaneously achieving high-accuracy physical analysis and high-resolution topology representation within the conventional single-resolution framework, a multiresolution topology optimization framework based on high-order finite elements is established. Through the decoupling of the design mesh and the analysis mesh, this framework develops the mapping mechanism from design variables to physical densities, together with the associated optimization formulations, numerical integration methods for stiffness matrices, and sensitivity analysis methods, and is further extended to stress-constrained topology optimization problems. The results show that this framework can simultaneously achieve structural analysis accuracy and topology representation

resolution under controlled computational cost, and exhibits good applicability to stress-constrained topology optimization problems.

For topology optimization modeling under complex local constraints and special material conditions, this dissertation further investigates a high-accuracy topology optimization method based on arbitrary-order Hu–Zhang mixed finite elements. Based on the Hellinger–Reissner mixed variational principle, a stress-displacement mixed formulation for linear elasticity, an arbitrary-order Hu–Zhang mixed finite element discretization framework, and the corresponding topology optimization formulations are systematically established. Through low-order stabilization techniques and a partial relaxation strategy for vertex stress continuity under complex boundary conditions, the applicability of the proposed method to unstructured meshes, low-order cases, and complex engineering boundaries is further enhanced. Corresponding numerical solution frameworks are then developed for typical problems, including compliance minimization, nearly incompressible materials, and local stress constraints. Numerical results show that the arbitrary-order Hu–Zhang mixed finite element method has clear advantages in alleviating volumetric locking and enhancing the reliability of local stress constraint evaluation.

To meet the unified implementation requirements of the aforementioned high-accuracy discretization models and optimization algorithms, this dissertation designs and implements a high-performance topology optimization software platform, SOPTX, based on FEALPy, an open-source intelligent CAX computing engine. Building upon FEALPy' s tensor abstraction and multi-backend mechanism, the platform constructs a domain-oriented layered architecture and a core modular system around the topology optimization loop, thereby realizing modular decoupling among structural analysis, sensitivity analysis, and optimization updates. At the technical level, the platform develops an efficient matrix assembly and operator caching mechanism based on analytical precomputation and tensor contraction, a sensitivity analysis implementation based on automatic differentiation, and a performance-portable execution mechanism for different backends, including NumPy, PyTorch, and JAX, as well as heterogeneous CPU/GPU devices. Representative numerical experiments further verify the correctness and computational efficiency of SOPTX in efficient matrix assembly and sensitivity analysis based on automatic differentiation, as well as its extensibility and computational performance under multi-backend and heterogeneous CPU/GPU execution. The platform provides unified, reproducible, and extensible software support for the various topology optimization models and numerical methods developed in this dissertation.

In summary, this dissertation addresses several key numerical issues in contin-

uum structural topology optimization, and establishes a systematic research line spanning high-order finite element discretization methods, multiresolution high-accuracy topology optimization, arbitrary-order Hu–Zhang mixed finite element modeling, and high-performance software implementation. In particular, this work forms a coherent research line running through high-accuracy discretization modeling, optimization under complex local constraints, and unified software implementation and validation. Covering mathematical models, discretization schemes, sensitivity analysis, software implementation, and numerical validation, the related studies establish a relatively complete research chain integrating theory, methodology, and implementation. The present work provides systematic theoretical, methodological, and software support for high-accuracy finite element discretization, the modeling of complex local constraints, and reproducible computation for engineering applications in continuum structural topology optimization.

Key Words: High-Order Finite Elements; Hu-Zhang Mixed Finite Elements; Multiresolution Topology Optimization; Stress-Constrained Topology Optimization; Topology Optimization Software Platform SOPTX

目录

摘要 I
Abstract III
第 1 章 绪论 1
第 2 章 拓扑优化模型与数值基础 2
第 3 章 任意次多单元族拉格朗日有限元拓扑优化比较研究 3
第 4 章 基于高阶有限元的多分辨率高精度拓扑优化 4
第 5 章 拓扑优化中的任意次胡张混合有限元方法研究 5
第 6 章 拓扑优化软件平台 SOPTX 的开发及应用 6
6.1 引言 6
6.2 FEALPy 基础与二次开发接口概述 7
6.2.1 FEALPy 简介与框架概述 7
6.2.2 SOPTX 对 FEALPy 的架构继承与领域化扩展 7
6.3 SOPTX 平台设计与总体架构 9
6.3.1 总体设计原则 9
6.3.2 总体分层架构 11
6.3.3 核心模块划分与职责 12
6.4 SOPTX 关键技术与实现细节 15
6.4.1 高效矩阵组装与算子缓存机制 15
6.4.2 自动微分驱动的灵敏度分析实现 19
6.4.3 性能可移植执行机制与后端一致性保障 20
6.5 使用教程与接口说明 21
6.5.1 安装与环境配置 22
6.5.2 基础使用流程 22
6.5.3 核心类接口与自定义扩展说明 28
6.6 数值算例与平台能力验证 36
6.6.1 算例设置与统一框架 36
6.6.2 高效矩阵组装的能力验证与性能评估 37
6.6.3 自动微分灵敏度的能力验证与性能评估 38
6.6.4 多后端与异构设备切换的能力验证与性能评估 40
6.7 本章小结 44
第 7 章 总结与展望 46

第 1 章 绪论

第 2 章　拓扑优化模型与数值基础

第 3 章　任意次多单元族拉格朗日有限元拓扑优化比较研究

第 4 章　基于高阶有限元的多分辨率高精度拓扑优化

第 5 章　拓扑优化中的任意次胡张混合有限元方法研究

第 6 章 拓扑优化软件平台 SOPTX 的开发及应用

6.1 引言

连续体结构拓扑优化从理论走向工程应用，不仅依赖严谨的数学模型与稳定的数值算法，也需要具备良好可扩展性与可持续演进能力的软件平台作为实现支撑。然而，现有拓扑优化工程实现通常仍面临两类突出瓶颈：一类来自软件架构设计，另一类来自底层计算效率。一方面，传统迭代流程往往将有限元分析、灵敏度计算与优化更新紧密耦合在一起，导致代码复用性较差；当引入新的物理场模型或更复杂的正则化策略时，常常需要进行侵入式修改，从而限制了算法研究、实现扩展与跨问题迁移的效率。另一方面，在大规模、高分辨率复杂结构设计中，频繁矩阵组装与大规模线性方程组求解通常构成主要计算开销。若缺乏统一的张量计算抽象与面向异构硬件的性能可移植执行机制，先进数值方法往往难以形成稳定、可复现且高效的工程计算流程。

为克服上述瓶颈，本章依托 FEALPy [? ?] 通用数值计算引擎，开发了面向拓扑优化的高性能模块化软件平台 SOPTX [?]。区别于传统基于过程式脚本的实现方式，本章以“分析–优化解耦”为核心设计原则，围绕拓扑优化迭代流程，将材料插值、物理场分析、正则化处理、灵敏度分析与优化更新抽象为标准化、可组合的功能模块。在此基础上，SOPTX 围绕迭代过程中的关键计算环节进行了系统设计：引入了基于解析预计算与张量收缩的高效矩阵组装及缓存机制，以提升高频装配场景下的计算效率；集成了基于自动微分的灵敏度分析实现，降低了复杂约束梯度求解的推导与工程实现成本；并依托统一张量抽象层，实现了面向跨后端（NumPy/PyTorch/JAX）与跨异构硬件（CPU/GPU）的可移植加速执行，为大规模拓扑优化问题的高效求解提供了实现基础。

基于上述软件平台，本章系统构建了涵盖建模、分析、优化与后处理的完整计算流程。针对多样化的结构设计需求，平台提供了统一而可扩展的调用接口，可支持柔顺度最小化、柔顺机构设计以及局部应力约束等典型模型；针对算法复现与二次开发需求，明确了核心类的模块化组装规则与自定义扩展机制。最后，通过若干基准测试与大规模三维数值算例，系统验证了平台在高效矩阵组装、自动微分灵敏度分析、多后端切换与异构硬件执行等方面的数值一致性、扩展能力与计算性能，为本文所提出的多类拓扑优化模型与数值方法提供了统一、可复现且可扩展的软件实现支撑。

6.2 FEALPy 基础与二次开发接口概述

本节简要概述 SOPTX 所依托的通用有限元计算底座 FEALPy 及其二次开发接口。首先介绍 FEALPy 的分层架构与后端管理机制，说明其在网格表示、函数空间构造、数值积分与矩阵装配以及线性方程组求解等方面为拓扑优化迭代提供的基础能力。随后从二次开发与接口适配的角度，系统梳理 SOPTX 在多后端张量算子抽象、通用组件复用、求解流程组织与统一调用入口等层面对 FEALPy 的继承与扩展关系，从而为后续 SOPTX 的总体架构设计与关键实现细节（如高效装配与缓存、基于自动微分的灵敏度计算以及跨后端与跨设备的可移植加速）建立清晰的依赖边界与技术背景。

6.2.1 FEALPy 简介与框架概述

FEALPy 是一套开源的智能计算引擎，围绕张量化计算组织数据结构与计算流程，旨在为科学与工程计算提供高效、灵活且易于扩展的通用软件底座。其总体设计采用分层式架构，依据“张量–共性–算法–应用”的依赖关系自下而上划分为张量层、共性层、算法层与领域层 [?]（如图 6.1所示）。其中，张量层负责屏蔽不同数值后端的差异，通过计算图（CGraph）等机制提供统一的数组与算子接口，从而支持跨后端与跨设备的可移植计算。其上，共性层提炼并集成了多类数值方法共享的基础组件与数据结构，包括网格表示、函数空间、数值积分与矩阵装配、稀疏线性代数与线性方程组求解等，为上层算法实现提供统一的离散对象与求解流程。进一步地，算法层在共性层对象之上实现典型离散与数值算法（如以有限元方法为核心的离散与求解流程等），并允许组合形成更复杂的计算模块。最后，领域层面向具体应用将算法模块与物理建模进行封装与组织，形成可复用的应用模型与示例算例，从而构成自底向上可组合、可扩展的数值仿真能力体系。

需要强调的是，在多后端与跨硬件适配方面，FEALPy 在张量层引入后端管理器（Backend Manager），为数组创建、广播、归约、爱因斯坦求和等常用张量算子提供了统一接口，并在运行时根据用户选定的计算后端调用其对应实现，从而实现“高层逻辑一次编写、多后端一致执行”。该张量抽象接口在设计上参考并遵循了 Python Array API Standard [?]，使框架能够以统一 API 对接 NumPy [?]、PyTorch [?]、JAX [?] 等主流张量计算后端。在此基础上，依托底层后端库对不同设备（CPU/GPU）的原生支持，FEALPy 能够在异构硬件环境中保持上层算法接口的严格一致，进而实现代码级的可移植执行与高性能加速。

6.2.2 SOPTX 对 FEALPy 的架构继承与领域化扩展

SOPTX 以 FEALPy 的通用数值计算体系为基础，面向拓扑优化“建模 → 离散 → 求解 → 灵敏度 → 更新”的迭代主链路，进一步抽象并固化各环节之间的数

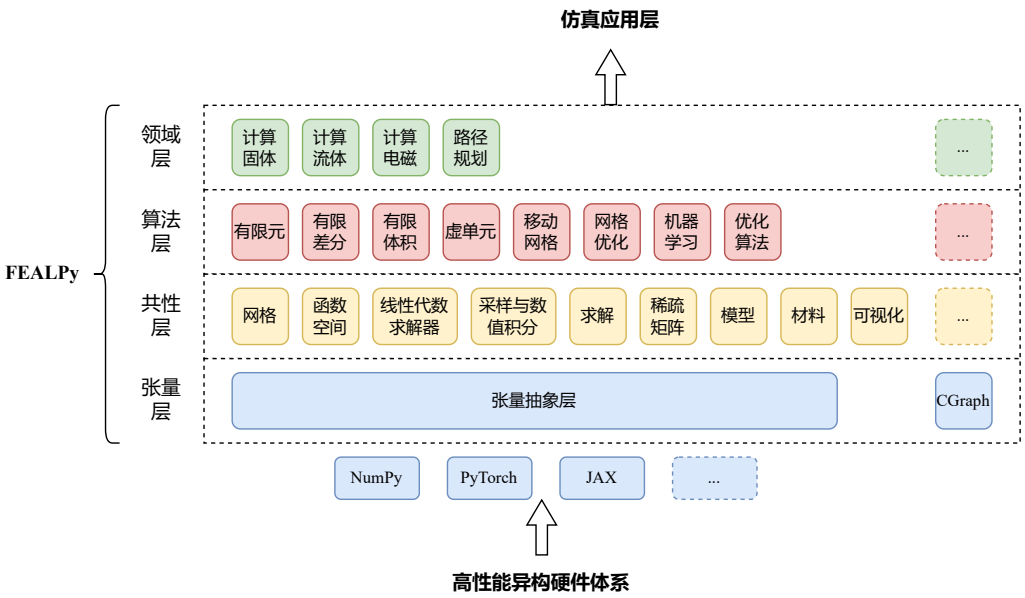


图 6.1 FEALPy 软件平台分层架构图

据对象与调用约定，形成可复用的二次开发接口。本节从可复用接口的角度，梳理 SOPTX 对 FEALPy 的对接关系与适配策略：按 FEALPy 自底向上的分层，将相关接口归纳为张量层、共性层、算法层与领域层四类，并分别说明 SOPTX 在每一层的复用模块、扩展封装与接口边界。

在张量层，FEALPy 通过后端管理器提供统一的张量抽象接口，使上层数值算子能够在 NumPy、PyTorch、JAX 等后端之间保持一致的调用方式。SOPTX 利用这一机制实现多后端切换与异构硬件适配，在目标与约束关于设计变量的计算过程可微的前提下，还可进一步调用 PyTorch/JAX 的自动微分机制，对目标函数与约束函数的梯度信息进行自动求导，从而大幅降低复杂约束梯度的推导与工程实现成本。

在共性层，FEALPy 提供网格、函数空间、采样与数值积分、稀疏数据结构、线性代数求解以及模型与材料等通用组件，为 SOPTX 中有限元分析模块提供直接支撑。具体而言，SOPTX 复用 FEALPy 的网格与几何实体管理接口完成设计域离散与测度计算，复用函数空间与自由度管理接口完成位移场离散与边界条件施加，并在此基础上组织数值积分与稀疏装配流程，以形成拓扑优化迭代中所需的离散方程组与后处理物理量。同时，SOPTX 复用 FEALPy 的材料参数与模型组织机制，将本构参数、载荷与边界条件等配置以统一数据结构纳入求解流程，为后续材料插值、目标与约束评估以及算例复现提供稳定的接口基础。

在算法层，FEALPy 提供面向多类数值方法的框架化组织方式以及求解流程相关的基础设施。SOPTX 主要复用其中与结构拓扑优化紧密相关的有限元分析与线性/非线性求解能力，将其作为“状态方程求解器”嵌入优化迭代；而拓扑优化特有

的更新策略（如 OC、MMA 等优化器）、正则化策略（过滤/投影）及约束处理等，则由 SOPTX 在平台层进行面向对象封装与统一接口管理，以保证算法可替换与流程可复用。

在领域层，FEALPy 面向典型学科应用提供问题入口与示例算例的组织方式，便于围绕特定物理场形成可复用的建模与求解工作流。SOPTX 以结构力学（尤其是线弹性）为主要应用入口，在 FEALPy 通用求解链路之上进一步封装拓扑优化任务级流程，实现材料插值模型（如 SIMP/RAMP）、体积分数与应力等约束形式，以及柔顺度最小化与柔顺机构设计等典型问题的算例配置、参数组织与结果后处理，从而形成面向研究复现与工程应用的可扩展实验体系。

综上，SOPTX 将 FEALPy 的张量抽象与多后端机制作为计算底座，复用其网格与函数空间、数值积分与稀疏装配、线性代数求解与求解流程组织等通用能力，从而以较低成本构建拓扑优化迭代所需的有限元分析主流程。在此基础上，SOPTX 面向拓扑优化的专用环节（材料插值、过滤与投影、目标与约束评估、优化器更新与工作流编排）进行模块化封装，实现算法组件的可替换与跨算例复用，为后续的平台总体架构与关键实现提供接口边界与依赖依据。

6.3 SOPTX 平台设计与总体架构

SOPTX 框架在 FEALPy 的分层体系中定位于“仿真应用层”，面向结构拓扑优化的建模、分析与优化一体化工作流。依托 FEALPy 提供的底层网格与几何处理能力、有限元离散与求解器体系及高性能张量计算后端管理机制，SOPTX 将拓扑优化迭代中的“密度映射 → 有限元分析 → 目标/约束评估与灵敏度计算 → 优化更新”等关键环节进行组件化封装与流程化编排。平台通过统一的数据对象与接口契约，严格规范了跨模块的信息传递，在确保算法模块高度解耦与易扩展的同时，为大规模问题的高效求解与多后端加速奠定坚实基础。

6.3.1 总体设计原则

从算法结构上看，变密度拓扑优化可抽象为稳定的迭代闭环：给定广义设计变量 $\boldsymbol{d}^k$，经映射与正则化得到用于分析的物理密度场 $\boldsymbol{\rho}^k = f(\boldsymbol{d}^k)$（该过程涵盖过滤、投影及参数连续化等操作）；随后，求解状态方程以获得状态变量 $\boldsymbol{s}^k$（在位移型离散下 $\boldsymbol{s} = \boldsymbol{u}$，而在混合型离散下则可扩充为 $\boldsymbol{s} = (\boldsymbol{\sigma}, \boldsymbol{u})$）；据此完成目标与约束的评估及其灵敏度计算后，最终由优化算法生成新的设计变量 $\boldsymbol{d}^{k+1}$。该迭代闭环既包含了矩阵组装与线性系统求解等计算密集型数值环节，又依赖正则化策略以抑制数值不稳定性并控制结构的最小特征尺度。由于不同物理模型、离散方法与优化算法在上述环节中的组合方式差异显著，若采用传统的紧耦合流程进行代码实现，将极大阻碍理论方法的扩展与底层算法的复用。基于此，SOPTX 确立了“迭代闭环结构

稳定、功能环节组件化解耦”的总体设计原则，将前述关键环节深度抽象为职责清晰、接口统一的策略组件，并通过标准化的数据对象与接口契约将其灵活编排为可重构的工作流，从而在确保底层数值计算稳定性的同时，兼顾了学术研究的快速迭代与工程应用的高效稳健执行。

图 6.2 给出了 SOPTX 面向拓扑优化闭环的组件化设计思想。闭环可分解为四类关键功能：映射与正则化、结构响应分析、目标/约束评估与灵敏度分析，以及优化更新。同时，平台将网格、有限元空间、数值积分与张量化装配、稀疏线性代数与求解器接口、后端管理等通用能力下沉至 FEALPy 支撑层，从而为上层拓扑优化工作流提供统一、可扩展的基础设施。

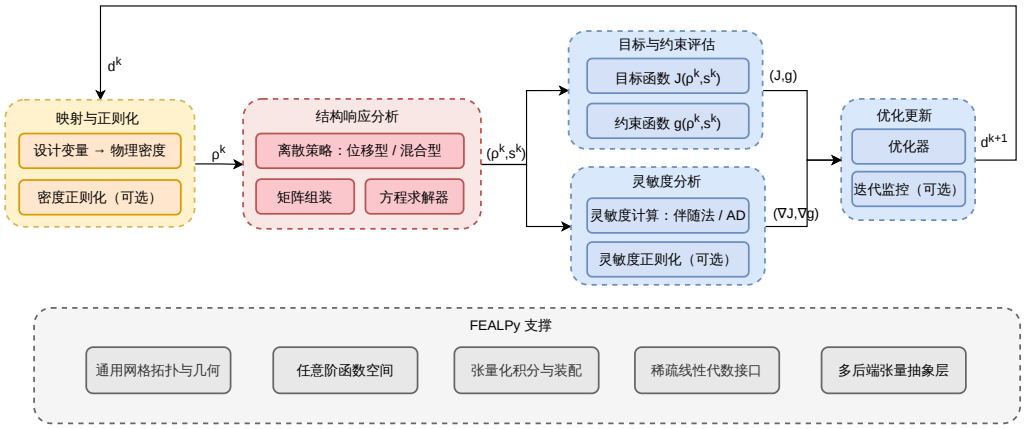


图 6.2 面向拓扑优化迭代闭环的组件化功能划分与数据流

在上述组件化设计思路指导下，SOPTX 的总体设计遵循以下五项核心原则：

1. **分析与优化解耦的分层原则**：平台严格划分有限元分析与优化更新的职责边界。分析模块专注于离散建模、积分装配与方程求解，并对外提供系统状态变量及必要的灵敏度接口；优化模块仅基于目标/约束函数值 (J, g) 及其梯度信息 $(\nabla J, \nabla g)$ 执行更新策略。这种解耦机制避免了优化器对具体物理模型或离散细节的直接依赖，确保了闭环流程在更换物理模型或求解策略时的鲁棒性。

2. **标准化接口的可扩展性原则**：平台统一了迭代闭环中的核心数据对象与接口契约，固定了设计变量 $\boldsymbol{d}$、物理密度 $\boldsymbol{\rho}$ 与状态变量 $\boldsymbol{s}$ 的张量化表达及传递规范，其中 $\boldsymbol{s}$ 可表示位移型（位移 $\boldsymbol{u}$）或混合型（应力–位移对 $(\boldsymbol{\sigma}, \boldsymbol{u})$）状态变量组。主循环仅负责组织 $\boldsymbol{d}^k \to \boldsymbol{\rho}^k \to \boldsymbol{s}^k \to (J, g, \nabla J, \nabla g)$ 的标准数据流，使得新增算子或物理模型只需遵循该接口契约即可实现“即插即用”式接入。

3. **几何拓扑与离散形式无关的通用抽象原则**：依托 FEALPy 提供的通用网格拓扑、测度计算与几何映射接口实现核心算子，从而降低对特定网格类型与编号规则的依赖。同时，将有限元空间构造与离散细节封装于分析模块内部，使上

层流程对单元阶次与离散方案保持透明，在接口不变的前提下支持计算策略的灵活切换与扩展。

4. **全流程可定制的策略组合原则**：将映射/正则化、材料插值、求解器配置及优化更新策略等关键环节深度封装为可替换组件。用户可根据特定问题类型与约束形式进行灵活的组合配置，而无需对平台主框架执行侵入式修改。

5. **后端无缝切换与自动微分协同原则**：依托 FEALPy 后端管理器构建统一张量抽象层，确保同一套高层优化逻辑能够在不同计算后端上一致执行。平台支持通过简单配置实现 CPU/GPU 异构硬件的无缝切换，并在支持自动微分的后端上提供自动微分计算路径，同时保留高性能解析灵敏度接口，以兼顾计算效率、开发便捷性与建模灵活性。

6.3.2 总体分层架构

在 FEALPy 的宏观架构体系中，SOPTX 定位于最顶端的“仿真应用层”，旨在面向结构拓扑优化问题提供一体化的建模与求解工作流。为了在这一层级内实现功能的灵活扩展与高效迭代，SOPTX 进一步构建了典型的高内聚、低耦合“三层软件架构”。如图 6.3 所示，该架构自上而下划分为用户交互层、核心算法层与基础后端层：上层负责问题描述与实验组织；中层承载拓扑优化的领域逻辑与闭环编排；下层则提供通用离散基础设施与多后端张量算子支撑。该分层设计将领域应用与通用数值计算底座进行了有效隔离，使得策略替换、模型扩展与异构硬件切换均可在清晰的层间接口约束下无缝完成。

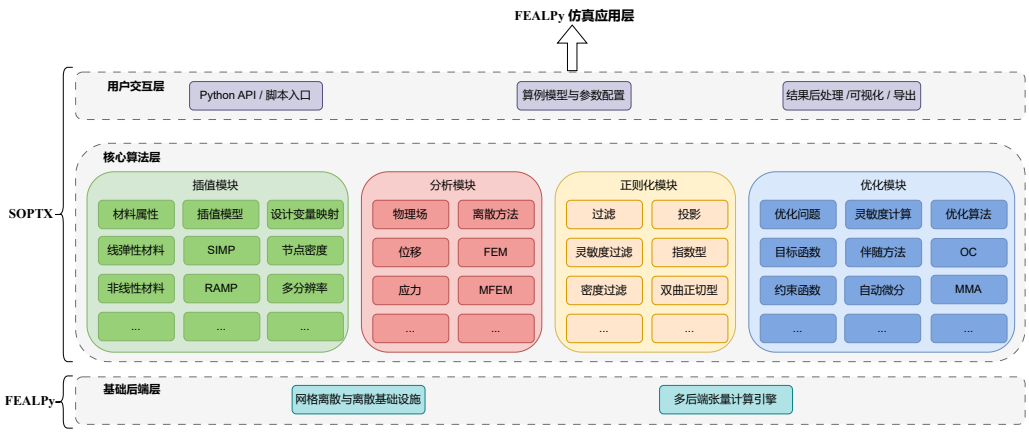


图 6.3 SOPTX 软件平台总体分层架构图

针对图 6.3 所展示的三层软件架构，各层级的具体组成与核心职责详细阐述如下：

1. **用户交互层**：位于架构顶层，是面向研究与工程使用的直接入口。该层提供标准化的 Python API 脚本调用接口，用于算例配置、参数管理与结果后处理（如数据可视化与模型导出）。用户可通过简洁的配置项自由选择组合不同优化策略与计算后端，而无需直接干预网格剖分、矩阵组装与方程求解等底层实现细节，极大降低了平台的使用门槛。

2. **核心算法层**：位于架构中层，是整个平台的算法中枢，负责组织拓扑优化迭代闭环所需的关键计算环节，并通过统一的数据对象与接口契约完成层内各组件的紧密协同。该层对上屏蔽了底层计算实现差异，对下适配通用离散算子与张量后端，从而确保了迭代闭环主流程在引入新算法策略或切换底层实现时具备极强的系统稳定性与鲁棒性。

3. **基础后端层**：位于架构底层，为上层算法提供几何离散与数值计算的双重支撑。一方面，该层沉淀了网格拓扑管理、函数空间构造、数值积分与张量化装配等有限元离散基础设施。另一方面，通过引入 FEALPy 的后端管理器提供统一的张量抽象与硬件适配机制，无缝支持 CPU/GPU 等异构硬件加速，并可在支持自动微分机制的后端（如 JAX 或 PyTorch）选择启用自动微分计算路径，为上层优化求解提供高效、可靠的导数计算支撑。

6.3.3 核心模块划分与职责

为支撑拓扑优化迭代过程中的关键计算环节，SOPTX 在核心算法层将领域逻辑组织为四个相互协同且接口解耦的功能模块：插值模块、分析模块、正则化模块与优化模块。这种模块化划分的核心目的，在于将材料插值与密度映射、物理场响应求解、数值稳定化处理以及设计变量更新策略分别封装为标准化的可替换组件，从而在不侵入全局主工作流的前提下，支持新物理模型、新计算策略与底层新实现的无缝接入。

插值模块 该模块主要由核心类 `MaterialInterpolationScheme` 及其派生类构成，是连接优化设计变量与物理场分析的关键桥梁。该模块不仅刻画密度场下材料本构参数的非线性插值规律，还负责将设计变量输出为分析所需的物理密度表征（涵盖单元密度与节点密度，并原生支持多分辨率架构下的网格解耦）。为兼顾计算效率与复杂材料模型的未来扩展需求，该模块深度采用抽象接口与策略注册机制来组织核心计算能力，具体体现在以下三个方面：

1. 材料属性复用与接口标准化：模块复用 FEALPy 的 `Material` 基类，统一封装了拉梅常数、弹性刚度矩阵及应变/应力计算等基础力学组件；同时，面向拓扑优化的高频迭代特征，提供标准化的材料评估接口，使其能够在给定物理

密度场下批量、高效地生成插值后的弹性张量，并在多后端计算环境中保持一致的调用规范，以适应迭代过程中频繁的材料评估与更新需求。

2. 基于装饰器的泛型插值架构：针对材料插值模型多样且易变的特点，模块引入了基于装饰器的变体分发机制，以统一的抽象接口无缝集成 SIMP、修正 SIMP 与 RAMP 等经典插值方案，实现了插值策略的“按需切换与即插即用”。若需引入前沿插值模型或非线性材料行为，用户仅需继承基础类并实现预定的接口契约即可完成扩展，全程无需对框架核心架构执行任何侵入式修改。

3. 高级映射与多物理量插值支持：在统一接口框架下，模块进一步沉淀了面向复杂结构设计的高级映射与插值能力。其一，原生支持 MTOP，实现设计变量网格与有限元分析网格的分辨率解耦，并输出一致的物理密度描述用于后续分析；其二，面向不可压缩或近不可压缩材料情形，支持对杨氏模量 E 与泊松比 ν 等本构参数进行独立的惩罚与插值控制；其三，针对局部应力约束问题，内置了应力插值函数及其精确的导数评估接口。

分析模块 该模块是 SOPTX 平台的数值计算引擎，基于 FEALPy 的网格拓扑、函数空间与底层装配能力进行了深度的领域化封装。其核心计算功能由基于位移型有限元的 `LagrangeFEMAnalyzer` 与基于胡张混合元的 `HuZhangMFEMAnalyzer` 及其派生类实现。在接收插值模块传递的物理密度场与材料本构信息后，该模块负责建立偏微分方程的离散模型，完成全局矩阵装配与线性系统求解，并输出状态变量及目标/约束评估所需的关键物理量。面向拓扑优化高频迭代中“反复装配–反复求解”的计算特征，该模块的设计重点高度集中于物理建模的可扩展性、底层执行的计算效率以及离散形式的通用性，具体体现在以下三个方面：

1. 离散算法与建模扩展接口：模块针对不同设计需求提供两类代表性离散路径。对于柔顺度最小化等刚度设计问题，`LagrangeFEMAnalyzer` 采用位移型有限元，以位移 $\boldsymbol{u}$ 为状态变量求解平衡方程，具备计算效率高的优势；针对高精度应力评估场景，`HuZhangMFEMAnalyzer` 引入胡张混合有限元，将应力 $\boldsymbol{\sigma}$ 与位移 $\boldsymbol{u}$ 作为独立未知量联立求解，显著提升了复杂拓扑界面处的评估可靠性。同时，模块确立了标准化的分析器抽象规范，支持后续通过派生类重载关键算子以无缝接入新物理场或离散方案。

2. 张量化装配与线性求解器接口：针对全局矩阵高频重建产生的计算瓶颈，模块构建了基于张量运算的高效装配路径，并依托 FEALPy 的多后端张量抽象，实现在 CPU/GPU 异构硬件及主流后端（如 NumPy、PyTorch、JAX）上的一致性调用，显著降低装配开销。同时，模块封装了后端无关的线性代数接口，支持直接法（如 MUMPS）与迭代法（如 CG）的按需切换。用户无需修改主体代码即可通过配置项灵活适配不同计算规模与硬件资源下的求解需求。

3. 几何适应性与离散通用性：依托 FEALPy 的通用拓扑数据结构与函数空间抽象，分析模块在空间维度、网格类型与逼近阶次上表现出极强的适应性：可覆盖二维与三维问题，兼容单纯形网格（如三角形、四面体）、张量积网格（如四边形、六面体）以及更一般的多边形/多面体网格；同时支持通过参数配置提升基函数的多项式阶次，以满足高精度分析需求。此外，模块封装了标准化的模型与边界条件接口，使载荷定义与约束施加尽可能独立于具体的网格拓扑与离散阶次，显著提升了拓扑优化建模与实验复现的通用性。

正则化模块 该模块是 SOPTX 平台中负责设计变量预处理与几何属性控制的核心组件，旨在消除拓扑优化中典型的数值不稳定性（如棋盘格现象与网格依赖性），并通过施加空间正则化约束来控制结构的最小特征尺寸，从而提升优化结果的可制造性。基于“分析与优化解耦”的设计原则，正则化模块以独立算子的形式接入迭代闭环：在物理密度映射路径上执行过滤与非线性投影操作，在梯度更新路径上对目标及约束灵敏度进行平滑处理。这种模块化的处理方式确保了在不侵入主流程逻辑的前提下，能够显著增强优化迭代过程的数值稳健性，具体体现在以下两个方面：

1. 基于 `KD-Tree` 的网格无关性与高效实现：区别于依赖规则网格模板的传统卷积滤波，正则化模块以物理空间邻域搜索为核心，引入 `KD-Tree` 等高效近邻查询机制，在任意离散点集上动态生成过滤加权矩阵。这一设计使正则化算子能够自然适配非结构化网格及复杂几何区域，彻底摆脱了对规则网格编号的依赖。针对大规模拓扑优化问题，模块结合稀疏存储结构与邻域信息预计算策略，大幅降低了高频迭代中过滤与投影操作的计算附加开销。

2. 多策略正则化算子与扩展接口：模块内置了三类核心正则化方法。其一为灵敏度过滤，通过对梯度场执行加权平均以抑制棋盘格现象；其二为密度过滤，通过对密度场进行空间平滑以缓解网格依赖性；其三为非线性投影，通过引入双曲正切型或指数型等光滑函数来逼近理想的 Heaviside 阶跃特征，旨在将平滑后的中间密度场映射为具有清晰边界的物理密度场，从而增强结构的 0-1 离散化倾向。此外，模块提供了统一的过滤基类接口，支持新方法通过继承实现核心映射算子来快速接入。

优化模块 作为 SOPTX 核心算法层的顶层驱动组件，优化模块负责将工程设计需求抽象为标准化的数学模型，并统筹设计变量的迭代更新。该模块严格遵循“分析与优化解耦”的设计原则，不直接依赖有限元网格拓扑或物理密度映射的底层细节，而是基于标准化的接口契约，仅以目标/约束函数值及其梯度信息 $(J, g_i, \nabla J, \nabla g_i)$ 为输入，驱动设计变量 $\boldsymbol{d}$ 的演化。这种高度抽象的数据流机制确保了在更换底层物理模型、离散机制或线性求解器时，全局优化主循环依然保持绝对稳定。该模块的核心架构设计体现在以下三个方面：

1. 优化问题的标准化定义与扩展接口：模块构建了统一的优化问题抽象描述，用于规范化目标函数 J 与约束函数 g_i 的计算逻辑，并内置了若干经典拓扑优化问题作为基准模板（如体积约束下的柔顺度最小化，以及包含局部物理量约束的复杂扩展形式）。该设计深度践行了“问题定义与求解流程相分离”的架构理念：用户只需继承基础问题类，并按接口契约实现目标与约束的函数值评估及灵敏度计算逻辑，即可将全新的优化任务无缝接入既有的迭代框架，从而大幅降低前沿理论复现与新问题建模的开发成本。

2. 解析梯度与自动微分双重灵敏度路径：为兼顾计算效率与建模灵活性，模块提供了解析伴随与自动微分双重获取机制。解析路径针对平台内置典型问题，基于离散伴随法求解全局梯度，并深度复用正向分析的线性系统分解结果以最小化额外开销。当选择支持自动微分的后端（如 JAX/PyTorch）时，模块可启用自动微分对正向计算过程执行端到端求导。该双重机制既支撑了复杂目标与新型约束的快速原型开发，也为解析灵敏度的正确性提供了一致性校验基准。

3. 优化算法库与可替换更新策略：模块以策略模式组织优化算法，实现“问题–算法”的深度解耦。当前内置两类代表性更新策略：OC 面向单约束刚度设计，具备轻量且收敛极快的优势；MMA 依托凸近似子问题，稳健处理多约束与一般非线性规划任务。同时，模块预留了标准的优化器扩展接口，支持无缝接入全局收敛移动渐近线法（Globally Convergent Method of Moving Asymptotes, GCMMA）等改进型算法或其它通用优化算法，以支持更复杂约束形式与更高维度的优化需求。

6.4 SOPTX 关键技术与实现细节

第 6.3 节确立了 SOPTX 平台“分析与优化解耦”的宏观架构。然而，要实现大规模复杂拓扑优化的高效求解，还必须突破底层数值计算链路的性能瓶颈。为此，本节将深入剖析平台的三大核心底层实现机制：首先，针对高频迭代中的全局矩阵重装配瓶颈，引入基于解析预计算与张量收缩的高效组装及缓存机制；其次，采用自动微分技术重构灵敏度分析框架，以降低复杂约束求导的实现成本并保障数值一致性；最后，依托统一张量抽象层，实现跨后端与跨异构硬件（CPU/GPU）的高性能可移植执行。这三项关键技术的协同，为平台的大规模高效计算提供了坚实保障。

6.4.1 高效矩阵组装与算子缓存机制

变密度拓扑优化通常采用“有限元分析–灵敏度分析–设计变量更新”的迭代闭环。在该框架下，材料插值使全局刚度矩阵 $\boldsymbol{K}(\boldsymbol{\rho})$ 的矩阵元素需随设计变量 $\boldsymbol{\rho}$ 的演

化而频繁更新，从而在每次迭代中都需要反复进行局部刚度贡献计算与全局稀疏装配（稀疏结构通常可复用）。尤其在三维问题或高阶单元离散情形下，位移自由度与积分点数显著增长，随着高阶单元局部自由度与所需积分精度呈非线性上升，导致单元数值积分、局部矩阵生成以及全局装配中的内存访问与索引映射开销更为突出，使矩阵组装成为有限元分析阶段的主要瓶颈之一。

为克服上述瓶颈，SOPTX 引入两类关键技术：一是将刚度矩阵计算中可复用的参考单元积分项与网格/几何映射相关量进行解耦分离。平台对参考单元上基函数及其导数相关的积分量进行预计算与缓存，并在装配阶段通过张量收缩实现局部矩阵的批量生成，最终完成全局稀疏矩阵的高效装配。二是在满足仿射映射与多项式被积函数等条件的场景下，进一步采用符号预积分策略对部分参考单元积分量给出精确表达。这不仅在保证一致性的同时降低了数值积分开销，还有效减少了积分误差对高阶离散稳定性的影响。

基于数值求积的快速组装

以标量拉普拉斯算子的双线性型为例，说明“单元无关–单元相关分离”的数学机制。设 $\hat{\Omega}$ 为参考单元，Ω_e 为物理单元，其映射关系 $F_e : \hat{\Omega} \to \Omega_e$ 记为 $\boldsymbol{x} = F_e(\hat{\boldsymbol{x}})$。令 $\hat{\phi}_i$ 为参考单元基函数，$\phi_i = \hat{\phi}_i \circ F_e^{-1}$ 为物理单元基函数，雅可比矩阵 J_e 及其行列式 $|J_e|$ 定义为：

$$J_e(\hat{\boldsymbol{x}}) = \frac{\partial \boldsymbol{x}}{\partial \hat{\boldsymbol{x}}}, \qquad |J_e| := |\det J_e|.$$

由链式法则可得梯度变换关系：

$$\nabla_{\boldsymbol{x}} \phi_i = J_e^{-T} \nabla_{\hat{\boldsymbol{x}}} \hat{\phi}_i.$$

引入第一基本形式（度量张量）：

$$G_e(\hat{\boldsymbol{x}}) = J_e(\hat{\boldsymbol{x}})^\top J_e(\hat{\boldsymbol{x}}),$$

则有如下恒等式：

$$(\nabla_{\boldsymbol{x}} \phi_i) \cdot (\nabla_{\boldsymbol{x}} \phi_j) = (\nabla_{\hat{\boldsymbol{x}}} \hat{\phi}_i)^\top G_e^{-1} (\nabla_{\hat{\boldsymbol{x}}} \hat{\phi}_j).$$

因此，在基于单元密度表征的材料惩罚模型下，单元刚度矩阵可表示为：

$$K_{ij}^{(e)}(\rho_e) = \mathcal{M}(\rho_e) \int_{\Omega_e} \nabla \phi_i \cdot \nabla \phi_j \, \mathrm{d}\boldsymbol{x} = \mathcal{M}(\rho_e) \int_{\hat{\Omega}} (\nabla_{\hat{\boldsymbol{x}}} \hat{\phi}_i)^\top G_e^{-1} (\nabla_{\hat{\boldsymbol{x}}} \hat{\phi}_j) \, |J_e| \, \mathrm{d}\hat{\boldsymbol{x}},$$

其中 $\mathcal{M}(\rho_e)$ 为依赖于设计变量的材料插值函数。由上式可以清晰地分离出三类贡献：

- 参考项（单元无关部分）：$\nabla_{\hat{\boldsymbol{x}}}\hat{\phi}_i$ 仅与参考单元、离散阶次以及所选求积规则有关，在初始化后可完全复用。
- 几何项（单元相关部分）：G_e^{-1} 与 $|J_e|$ 仅由单元几何映射决定，反映了物理单元的几何形变。
- 材料项（迭代相关部分）：$\mathcal{M}(\rho_e)$ 仅由优化迭代中的设计变量决定。

对于单纯形单元，在常见的仿射映射情形下，由于 J_e 与 G_e 在单元内为常量，可将其移出积分项，进而得到

$$K_{ij}^{(e)}(\rho_e) = \mathcal{M}(\rho_e)\,|J_e| \sum_{k,l=1}^{d} (G_e^{-1})_{kl}\,\hat{S}_{ij}^{kl}, \qquad \hat{S}_{ij}^{kl} := \int_{\hat{\Omega}} \frac{\partial\hat{\phi}_i}{\partial\hat{x}_k}\,\frac{\partial\hat{\phi}_j}{\partial\hat{x}_l}\,\mathrm{d}\hat{\boldsymbol{x}}.$$

其中，$\hat{S}_{ij}^{kl}$ 是仅由参考单元类型、离散阶次与所选求积规则决定的参考单元四阶张量。该张量可在初始化阶段一次性预计算，并在后续所有迭代中跨单元复用。在优化迭代过程中，仅需根据当前设计变量更新标量系数 $\mathcal{M}(\rho_e)$，并通过向量化张量收缩即可快速获取局部矩阵。

上述解耦与批量化思想同样适用于其他单元类型与映射情形。对于张量积单元，在规则矩形/长方体网格或仿射几何映射情形下，J_e 与 G_e 同样为常量。此时，局部刚度矩阵呈现出“几何因子–参考张量”的解耦形式：通过预先缓存参考单元上的 $\hat{S}_{ij}^{kl}$，装配阶段仅需计算单元相关的几何量（如 $|J_e|$ 或 G_e^{-1}），并结合当前材料系数 $\mathcal{M}(\rho_e)$ 执行向量化张量收缩，最终完成全局稀疏矩阵的批量化装配。此外，若采用一般等参映射（曲边或高阶几何映射），雅可比矩阵 $J_e(\hat{\boldsymbol{x}})$ 将随积分点位置变化。此时需引入求积点维度，并对参考单元的“导数–权重”张量进行缓存。尽管此情形下的计算复杂度有所增加，但其本质与本文所述的“单元相关项–单元无关项”分离思想一致，在此不再展开。

基于解析预计算的快速组装

在上述分离框架中，参考单元张量 $\hat{S}_{ij}^{kl}$（或在等参映射情形下引入求积点索引的 $\hat{S}_{ij}^{kl,q}$）通常依赖高阶数值求积获得。为进一步降低初始化阶段的计算代价，并在满足条件时避免数值积分误差（仅保留浮点舍入误差），SOPTX 进一步实现了参考单元的解析预计算与结构化缓存策略：在参考单元上将基函数及其导数相关的积分项尽可能转化为多项式的闭式积分，或利用张量积结构进行降维处理，从而生成精确的可复用参考算子，供后续所有迭代直接调用。

对于单纯形单元，k 次拉格朗日基函数的重心坐标构造、晶格节点与编号规则已在第 **??** 节给出。基于该构造，基函数乘积 $\hat{\phi}_i\hat{\phi}_j$ 以及导数乘积 $\partial_{\hat{x}_k}\hat{\phi}_i\,\partial_{\hat{x}_l}\hat{\phi}_j$ 均可展开为重心坐标单项式 $\prod_{r=0}^{d}\lambda_r^{I_r}$ 的有限线性组合。从而 $\hat{S}_{ij}^{kl}$ 的计算可归结为重心坐标

单项式的解析积分。对任意 d 维单纯形 τ 及多重指标 $\boldsymbol{I} = (I_0, \ldots, I_d)$，有经典闭式积分公式 [?]：

$$\int_\tau \lambda_0^{I_0} \lambda_1^{I_1} \cdots \lambda_d^{I_d} \, \mathrm{d}\boldsymbol{x} = \frac{d! \, \boldsymbol{I}!}{(|\boldsymbol{I}| + d)!} \, |\tau|, \qquad \boldsymbol{I}! := \prod_{r=0}^{d} I_r!, \quad |\boldsymbol{I}| := \sum_{r=0}^{d} I_r.$$

基于此，SOPTX 在初始化阶段将被积函数自动展开为重心坐标单项式的线性组合，利用上述闭式公式精确计算 $\hat{S}_{ij}^{kl}$ 并建立结构化缓存。该策略在单纯形仿射映射情形下尤为高效：由于几何映射相关量在单元内为常量，装配阶段仅需读取预计算的参考张量并与几何因子执行张量收缩，即可直接生成局部矩阵，从而彻底避免了在每次迭代中对同一参考积分进行重复数值求积的计算开销。

对于张量积单元，参考单元定义为超立方体 $\hat{\Omega} = \hat{I}^d$，其 k 阶拉格朗日基函数由一维基函数的张量积构造。例如三维情形 $\phi_{ijk}(\xi, \eta, \zeta) = \phi_i(\xi)\phi_j(\eta)\phi_k(\zeta)$。因此，参考单元上由基函数及其导数组成的积分项可分解为一维积分的乘积（Kronecker 积组合）。SOPTX 利用该可分离结构，在初始化阶段仅需对一维参考区间上的“基函数/导数–权重”张量进行预计算与缓存，并在高维情形下通过张量积结构进行组合复用，从而以更低的计算与存储代价生成所需参考算子。对于多项式基函数，一维积分项既可采用闭式积分得到精确表达，也可通过高精度数值求积预先生成，无论采用哪种方式，在装配阶段均只需执行张量收缩运算。

两类快速组装在实现层面遵循统一的“预计算–迭代复用”范式：在初始化阶段生成参考单元算子（可由高阶数值求积得到，亦可由解析/符号预计算得到），并以“单元类型—离散阶次—导数方向”等信息作为索引参数建立结构化缓存；在优化迭代阶段不再重复构造参考算子，而仅在单元（或积分点）层面更新几何相关量（如 $|J_e|$、G_e^{-1} 等）与材料插值系数 $\mathcal{M}(\rho_e)$，随后通过向量化张量收缩与批量化稀疏累加生成局部矩阵并高效装配到全局系统。为适配多后端计算环境，缓存张量的数据布局与张量算子接口需与平台底层抽象保持一致，从而保证参考算子在不同计算后端间可无缝复用。

以上两类加速策略不仅适用于标量拉普拉斯型算子，同样可自然推广至线弹性等向量场问题。对于线弹性问题，梯度变换关系被融入应变–位移矩阵 $\boldsymbol{B}$ 的构造过程中，其局部刚度矩阵为

$$\boldsymbol{K}_e = \int_{\Omega_e} \boldsymbol{B}^\top \boldsymbol{D}(\rho_e) \boldsymbol{B} \, \mathrm{d}\boldsymbol{x},$$

该积分形式同样满足“材料系数–几何映射–参考算子”的解耦分离结构。因此，平台可直接复用前述两类快速组装机制中的张量收缩与批量化装配流程，实现线弹性刚度矩阵的高效生成。

6.4.2 自动微分驱动的灵敏度分析实现

灵敏度分析是变密度拓扑优化迭代更新的核心环节，其精度与一致性直接影响优化算法的收敛性与最终构型的可靠性。传统开发通常采用“手动推导伴随方程 + 手工实现梯度算子”的方式：通过伴随变量避免显式计算 $\mathrm{d}\boldsymbol{U}/\mathrm{d}\boldsymbol{\rho}$，将全导数重写为可计算的形式。然而，随着密度映射机制（如非线性材料插值、投影）与物理约束（如局部应力聚合、多物理场耦合）的日益复杂，手动推导与维护梯度算子将显著增加开发成本，且极易引入计算错误。为此，SOPTX 在灵敏度模块中引入自动微分技术，旨在构建一个可复用、高扩展且一致性更强的灵敏度计算框架。

在具体实现层面，SOPTX 依托统一张量抽象层对接底层算子库。当选用具备自动微分能力的后端（如 PyTorch 或 JAX）时，平台可利用其反向传播机制，自动完成目标与约束函数关于设计变量的导数信息传播。同时，为兼顾极端规模下的计算效率与内存限制，平台保留了传统的解析伴随路径。该双重路径设计不仅保障了在算子不可微或后端受限时的平滑回退，也为自动微分结果的数值正确性提供了严格的交叉校验基准。

在变密度拓扑优化中，可将带有平衡方程约束的优化模型抽象为如下参数优化问题：

$$\min_{\boldsymbol{d}} \mathcal{J}(\boldsymbol{\rho}(\boldsymbol{d}), \boldsymbol{U}(\boldsymbol{d})), \quad \text{s.t.} \quad \boldsymbol{F}(\boldsymbol{U}, \boldsymbol{\rho}(\boldsymbol{d})) = \boldsymbol{0},$$

其中 $\boldsymbol{F}(\boldsymbol{U}, \boldsymbol{\rho}) = \boldsymbol{0}$ 表示离散状态方程（对于线弹性问题，$\boldsymbol{F} = \boldsymbol{K}(\boldsymbol{\rho})\boldsymbol{U} - \boldsymbol{f}$）。根据链式法则，目标函数对设计变量的全导数可表示为：

$$\frac{\mathrm{d}\mathcal{J}}{\mathrm{d}\boldsymbol{d}} = \frac{\partial \mathcal{J}}{\partial \boldsymbol{\rho}} \frac{\partial \boldsymbol{\rho}}{\partial \boldsymbol{d}} + \frac{\partial \mathcal{J}}{\partial \boldsymbol{U}} \frac{\mathrm{d}\boldsymbol{U}}{\mathrm{d}\boldsymbol{d}}.$$

由于状态变量 $\boldsymbol{U}$ 由状态方程隐式确定，直接计算全导数雅可比矩阵 $\mathrm{d}\boldsymbol{U}/\mathrm{d}\boldsymbol{d}$ 的代价极其高昂。对状态方程两侧求导，可得隐式微分关系：

$$\frac{\partial \boldsymbol{F}}{\partial \boldsymbol{U}} \frac{\mathrm{d}\boldsymbol{U}}{\mathrm{d}\boldsymbol{d}} + \frac{\partial \boldsymbol{F}}{\partial \boldsymbol{\rho}} \frac{\partial \boldsymbol{\rho}}{\partial \boldsymbol{d}} = \boldsymbol{0} \implies \frac{\mathrm{d}\boldsymbol{U}}{\mathrm{d}\boldsymbol{d}} = -\left(\frac{\partial \boldsymbol{F}}{\partial \boldsymbol{U}}\right)^{-1} \frac{\partial \boldsymbol{F}}{\partial \boldsymbol{\rho}} \frac{\partial \boldsymbol{\rho}}{\partial \boldsymbol{d}}.$$

传统伴随法通过引入伴随向量 $\boldsymbol{\lambda}$，令其满足伴随方程 $\left(\frac{\partial \boldsymbol{F}}{\partial \boldsymbol{U}}\right)^{\top} \boldsymbol{\lambda} = \left(\frac{\partial \mathcal{J}}{\partial \boldsymbol{U}}\right)^{\top}$，从而将目标函数的梯度等价转化为：

$$\frac{\mathrm{d}\mathcal{J}}{\mathrm{d}\boldsymbol{d}} = \left(\frac{\partial \mathcal{J}}{\partial \boldsymbol{\rho}} - \boldsymbol{\lambda}^{\top} \frac{\partial \boldsymbol{F}}{\partial \boldsymbol{\rho}}\right) \frac{\partial \boldsymbol{\rho}}{\partial \boldsymbol{d}}.$$

对于线弹性静力学问题，雅可比矩阵 $\partial \boldsymbol{F}/\partial \boldsymbol{U} = \boldsymbol{K}(\boldsymbol{\rho})$，伴随方程与原物理问题的状态方程共享相同的算子结构，这保证了导数计算的一致性与高效性。

自动微分的引入并不改变灵敏度分析的数学本质，而是在实现层面通过“算子

化”方式自动完成链式法则的组合，从而避免了在复杂目标、约束或映射组合下冗余的手工梯度推导。具体而言，SOPTX 将目标函数、约束函数与密度映射等环节抽象为一系列可组合的算子。在统一的张量接口下，诸如 $\partial\mathcal{J}/\partial\boldsymbol{\rho}$、$\partial\boldsymbol{F}/\partial\boldsymbol{\rho}$ 及 $\partial\boldsymbol{\rho}/\partial\boldsymbol{d}$ 等中间项均可由自动微分机制自动追踪并计算，确保了梯度信息与离散前向计算之间的数值精确性与严格一致性。

需要强调的是，状态变量 $\boldsymbol{U}$ 通常由大型线性系统求解获得。若将求解过程（如迭代法的每一步残差更新）直接展开并记录于计算图中，将引入巨大的内存开销与算子冗余。为此，平台采用了隐式微分实现策略：将线性求解视为一个算子 $\boldsymbol{U} = \mathcal{S}(\boldsymbol{K}(\boldsymbol{\rho}), \boldsymbol{f})$，其反向传播过程并不回溯求解器内部的迭代细节，而是直接通过求解前述伴随方程来获取向量–雅可比积。这一处理在数学上与伴随法完全等效，但在工程实现上实现了物理求解器与自动微分引擎的逻辑解耦，极大地提升了大规模拓扑优化问题的求导效率与后端复用能力。

在数值实现层面，SOPTX 依托 FEALPy 的张量化算子体系，将“局部核函数定义”提升为可在大规模网格上高效执行的批处理计算，形成了“Value-and-Grad”的统一调用范式。其核心实现包含以下两个关键维度：

1. 核函数自动求导：针对用户定义的标量核函数（如材料插值函数或局部应力准则），平台通过后端自动微分算子生成其对应的导数计算规则。这使得非线性材料本构、复杂的应力失效准则等导数逻辑能够得到自动化维护，极大地降低了算法扩展的门槛。

2. 批量向量化映射：通过向量化接口将单元级梯度提升为面向全域网格的批处理算子。该机制规避了 Python 解释器层面的显式循环开销，并允许后端通过 JIT（即时编译）、算子融合与并行内核调度，在多核 CPU 或 GPU 上实现高效的硬件映射。这确保了 SOPTX 在保持代码结构简洁的同时获得较高的计算性能。

基于上述机制，SOPTX 的优化迭代可在不暴露伴随方程细节的前提下完成目标/约束的同步求值与梯度获取：优化模块仅以 $\mathcal{J}$，$\nabla\mathcal{J}$ 及约束对偶信息为输入驱动设计变量更新，而分析模块继续负责状态方程求解与必要的物理量后处理，从而保持“分析–优化解耦”的整体架构一致性，并为后续在不同物理场、不同离散形式与不同计算后端上的扩展提供稳定接口基础。

6.4.3 性能可移植执行机制与后端一致性保障

SOPTX 平台的高性能与跨平台执行能力，深度依托于对 FEALPy 底层张量抽象机制的复用与领域化适配。如第 6.2 节所述，FEALPy 通过后端管理器屏蔽了 NumPy、PyTorch 与 JAX 等底层数值计算库的语法差异。为此，SOPTX 在核心架

构上摒弃了算法层对具体数值库的直接依赖，严格遵循统一的张量接口规范。在具体实现中，平台将拓扑优化迭代中的关键算子（包括 SIMP/RAMP 材料插值、基于 `KD-Tree` 的邻域搜索与过滤权重矩阵构建，以及 OC/MMA 优化更新策略等）进行了全链路的张量化重构。通过这种深度封装，高层优化逻辑被转化为后端无关的张量化计算链路。在用户切换计算后端时，SOPTX 可自动将张量收缩、向量更新等领域算子动态分发至对应的底层实现。该设计不仅消除了针对不同硬件重复编写代码的冗余，更使得拓扑优化任务能够无缝继承底层数值库的性能优化。

在利用 GPU 等异构设备进行加速时，主机（Host）与设备（Device）之间频繁的数据传输往往成为制约整体效率的性能瓶颈。针对拓扑优化高频迭代与数据强耦合的特征，SOPTX 复用了 FEALPy 框架底层的数据驻留机制，并在此基础上构建了面向优化闭环的显存管理策略。在初始化阶段，平台调用 FEALPy 接口，将网格拓扑、参考单元积分算子及初始设计变量一次性推送至设备显存；在随后的数百步优化迭代闭环中，所有的物理场求解、灵敏度分析与变量更新均直接在设备端调用原生算子完成。这种封闭式循环封装确保了迭代主流程中无任何不必要的 Host-Device 通信，仅在需输出监控指标或可视化结果时回传少量标量数据，从而打通了 GPU 高性能计算的数据链路。

此外，针对不同后端库在默认浮点精度与运算策略上存在差异的问题，SOPTX 在调用 FEALPy 接口时强制约束了双精度算术规范。平台在算法架构层显式声明了所有核心中间变量（如物理密度场、梯度场）的数据类型，并通过统一的求解器接口向底层传递严格的收敛容差。这种严苛的数值控制策略，不仅保障了在多后端切换下优化轨迹的严格一致性，还有效抑制了 GPU 大规模并行累加时因运算顺序差异导致的微小舍入误差发散，为复杂拓扑构型的可靠演化提供了坚实的数值基础。

6.5 使用教程与接口说明

优秀的科学计算软件架构不仅需要高效的底层数值内核，还应具备规范的应用程序接口定义与高扩展性的领域建模逻辑。本节将通过具体的部署指南与典型算例，系统展示 SOPTX 平台的使用流程与接口特性。作为一款基于 FEALPy 框架构建的拓扑优化计算平台，SOPTX 深度践行了“分析与优化解耦”的设计原则，利用其高度模块化的组件架构与多计算后端的无缝切换能力，极大地降低了复杂拓扑优化问题的定义门槛与求解实现成本。下文将首先介绍开发环境的配置与依赖项管理，随后以端部受载的二维悬臂梁柔顺度最小化问题为例，详细解析从物理模型构建、离散策略配置到结果可视化的标准化工作流，从而直观体现该平台在面向前沿算法研究与工程实际应用中的便捷性与高效性。

6.5.1 安装与环境配置

SOPTX 平台完全基于 Python 语言开发，深度依托了其在科学计算与数据处理领域的丰富开源生态。为兼顾数值计算的稳定性与前沿算法开发的灵活性，平台推荐采用基于源码构建的方式进行部署。

作为 SOPTX 的底层支撑，开源智能 CAX 计算引擎 FEALPy 提供了核心的离散与求解能力。考虑到拓扑优化研究中频繁涉及核心算法的二次开发与底层调试，强烈建议用户在隔离的虚拟环境中配置开发环境，以有效避免项目间的包依赖冲突并保障运行环境的纯净性。具体的部署流程主要包含以下两个关键步骤：

1. 依赖项获取：利用分布式版本控制系统 Git，从远程代码仓库克隆 FEALPy 及 SOPTX 的最新源代码。

2. 可编辑模式安装：采用 Python 包管理工具（`pip`）的“可编辑模式”（附加 `-e` 参数）执行安装。该模式会在 Python 库路径中创建指向本地源码目录的软链接，而非进行静态的文件复制。这确保了用户对本地源码的任何侵入式修改均能在下一次运行时即时生效，从而提升了算法原型研发与调试的迭代效率。

标准的部署脚本与执行逻辑如代码 6.1 所示：

代码 6.1 SOPTX 及 FEALPy 源码安装脚本

```
# 1. 克隆并安装核心依赖库 FEALPy
git clone https://github.com/weihuayi/fealpy.git
cd fealpy
pip install -e .

# 2. 克隆并安装 SOPTX 平台
git clone https://github.com/weihuayi/soptx.git
cd soptx
pip install -e .
```

执行上述指令后，系统将自动解析并安装 NumPy、SciPy 等基础数学库依赖，进而配置出完整的拓扑优化计算环境。关于 SOPTX 及 FEALPy 的详细安装文档与源代码，读者可通过官方代码托管仓库 ① 进行查阅。

6.5.2 基础使用流程

为了直观展示 SOPTX 平台从数学建模到代码求解的完整映射流程，本节选取拓扑优化领域最为经典的基准问题——端部角点受载的二维悬臂梁作为演示对象。需要说明的是，本算例虽然在几何形式上与第五章中的悬臂梁相似，但两者的载荷

① SOPTX 与 FEALPy 项目地址分别为：`https://github.com/brighthe/soptx` 及 `https://github.com/weihuayi/fealpy`。

工况存在显著区别：第五章采用了右端中点受载以验证特定算法性能，而本节为了全面验证 SOPTX 平台与标准文献结果的一致性，采用了更为通用的右下角顶点受载标准工况。这种设置便于将平台计算结果与经典文献 [?] 的基准解进行直接对标。

算例 6.1：端部角点受载的二维悬臂梁

算例的物理模型与边界条件如图 6.4 所示。设计域设定为几何尺寸为 $L \times 0.625L$ 的矩形区域，其中特征长度取 $L = 160\,\mathrm{mm}$。设计域左边界施加固支约束，载荷施加于设计域右下角顶点处，为竖直向下的单位集中力 $P = 1\,\mathrm{N}$，其余边施加自由边界条件。假定实体材料为各向同性线弹性介质，杨氏模量 $E = 1\,\mathrm{MPa}$，泊松比 $\nu = 0.3$；优化目标是在满足体积分数 $V_f = 0.4$ 的约束下最小化结构柔顺度。求解过程中采用修正的 SIMP 插值模型，惩罚因子设定为 $p = 3$，灵敏度过滤半径设为 $r_{\min} = 6.0\,\mathrm{mm}$。

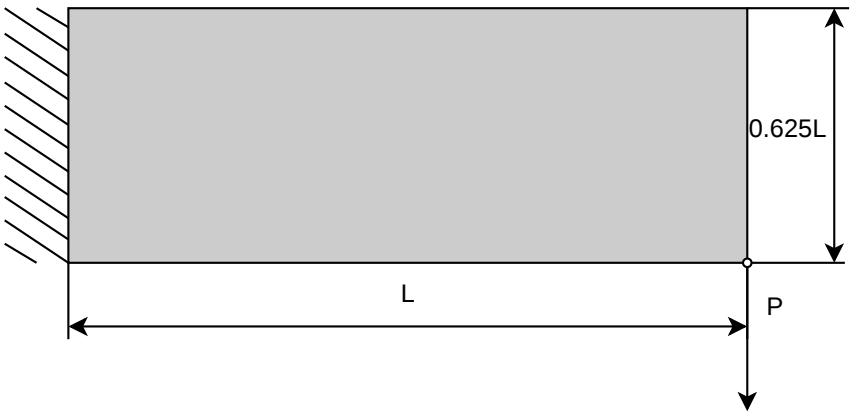


图 6.4 端部角点受载的二维悬臂梁设计域、载荷与边界条件示意图

利用 SOPTX 平台内置的参数化模型库，用户首先需要构建一个包含几何信息、网格拓扑及边界条件的物理模型对象。在本算例中，仅需调用 `Cantilever2dCorner` 类并指定设计域的几何边界参数（160×100）及角点载荷大小（$P = -1$），即可完成模型的初始化工作。随后，显式定义离散化策略：本算例通过 `init_mesh` 接口将单元类型设定为四边形单元，并将设计域剖分为 160×100 的结构化网格，最终生成用于有限元分析的网格对象，具体的实例化与调用过程如代码 6.2 所示。

代码 6.2 二维悬臂梁物理模型实例化与网格构建

```
from soptx.model.cantilever_2d_lfem import Cantilever2dCorner
# 1. 实例化物理模型：定义设计域 [0, 160]x[0, 100] 及载荷 P=-1
pde = Cantilever2dCorner(domain=[0, 160, 0, 100], P=-1)
# 2. 配置并生成网格：采用均匀四边形单元，离散规模 160x100
pde.init_mesh.set('uniform_quad')
```

```
mesh = pde.init_mesh(nx=160, ny=100)
```

在完成几何模型与离散网格的实例化后，需定义结构的本构关系及核心的材料插值模型。首先，构建基础材料对象：针对本算例的二维特性，采用各向同性线弹性模型，传入相应的本构参数数值（如杨氏模量设定为 1.0，泊松比设定为 0.3），并显式指定平面应力假设。随后，配置材料属性插值方案，建立设计变量（单元密度）与物理属性（杨氏模量）之间的映射关系。代码中采用了修正 SIMP 模型（`'msimp'`），并设置极小的孔洞模量参数 $E_{\min} = 10^{-9}$ 以避免刚度矩阵奇异性，具体的材料模型构建与插值方案配置如代码 6.3 所示：

代码 6.3 材料模型构建与插值方案配置

```python
from soptx.interpolation.linear_elastic_material import
    IsotropicLinearElasticMaterial
from soptx.interpolation.interpolation_scheme import
    MaterialInterpolationScheme
# 1. 定义基础材料属性: E=1.0, nu=0.3, 采用平面应力假设
material = IsotropicLinearElasticMaterial(
    youngs_modulus=1.0,
    poisson_ratio=0.3,
    plane_type='plane_stress'
)
# 2. 配置插值方案: 基于单元密度的修正 SIMP 插值
interpolation_scheme = MaterialInterpolationScheme(
    # 将设计变量定义在网格单元中心
    density_location='element',
    # 采用修正 SIMP 方法以避免刚度矩阵奇异性
    interpolation_method='msimp',
    options={
        'penalty_factor': 3.0,        # 惩罚因子 p=3
        'void_youngs_modulus': 1e-9,  # 空洞材料模量 Emin
        'target_variables': ['E'],    # 插值对象为杨氏模量
    },
)
```

为了实现结构平衡方程 $\boldsymbol{KU} = \boldsymbol{F}$ 的离散求解，需实例化拉格朗日有限元分析器 `LagrangeFEMAnalyzer`。作为 SOPTX 平台的数值计算引擎，分析器承担了“状态方程求解器”的核心职责。需要强调的是，该分析器的构建严格依赖于前文定义的材料模型与插值方案。这种深度绑定机制确保了在组装全局刚度矩阵时，能够准确引入拓扑设计变量（单元密度）对物理本构属性的修正作用。在具体的数值离散配置上，本算例选取 1 阶拉格朗日基函数以兼顾计算效率，并匹配 2 阶高斯求积方案以保障刚度矩阵的数值积分精度。同时，平台配置了标准的全局矩阵组装策略，

并指定具有高度数值鲁棒性的 MUMPS 直接求解器执行线性方程组的求解工作。有限元分析器的具体实例化过程如代码 6.4 所示。

代码 6.4 位移型有限元分析器的实例化配置

```
from soptx.analysis.lagrange_fem_analyzer import LagrangeFEMAnalyzer
# 构建有限元分析器
langrange_analyzer = LagrangeFEMAnalyzer(
    disp_mesh=mesh,                                # 位移网格
    pde=pde,                                       # 物理模型
    material=material,                             # 基础材料属性
    interpolation_scheme=interpolation_scheme,     # 拓扑插值方案
    space_degree=1,                  # 空间离散: Q1 单元
    integration_order=2,             # 数值积分: 2 阶高斯积分
    assembly_method='standard',      # 组装方式: 标准刚度矩阵组装
    solve_method='mumps',            # 求解器: MUMPS 直接求解器
)
```

基于已构建的位移型有限元分析器，下一步需要将抽象的物理描述转化为可求解的标准数学规划模型。具体而言，首先调用 ComplianceObjective 类定义柔顺度最小化目标函数，该算子通过绑定分析器实例，实现对系统全局刚度与位移场信息的实时获取，并完成相应的灵敏度计算。同时，实例化 VolumeConstraint 类以施加材料用量约束，本算例将设计域的材料体积上限设定为 40%。此外，为有效抑制拓扑优化离散求解中典型的棋盘格效应与网格依赖性等数值不稳定问题，显式引入 Filter 类构建正则化算子，并指定采用灵敏度过滤技术，将其过滤半径参数配置为 $r_{\min} = 6.0$。上述目标函数、约束函数及正则化算子的具体定义如代码 6.5 所示：

代码 6.5 优化模型定义：目标函数、约束函数及正则化算子

```
from soptx.optimization.compliance_objective import
    ComplianceObjective
from soptx.optimization.volume_constraint import VolumeConstraint
from soptx.regularization.filtering import Filter
# 1. 目标函数: 最小化结构柔顺度
compliance_objective = ComplianceObjective(analyzer=
    langrange_analyzer)
# 2. 约束条件: 设定目标体积分数为 40%
volume_constraint = VolumeConstraint(
    analyzer=langrange_analyzer,
    volume_fraction=0.4
)
# 3. 正则化设置: 构建灵敏度滤波器以抑制数值不稳定性
filter_regularization = Filter(
    design_mesh=mesh,            # 设计变量网格
```

```
    filter_type='sensitivity',  # 采用灵敏度过滤
    rmin=6.0,                   # 过滤半径 r=6.0
)
```

最后，针对单约束柔顺度最小化问题，构建基于 OC 的优化器实例。在求解配置上，设定最大迭代次数为 200，收敛容差（即相邻两次迭代间设计变量的最大无穷范数差）为 1×10^{-2}。为保证迭代过程的数值稳定性，显式引入了设计变量的移动极限（$m = 0.2$）与阻尼系数（$\eta = 0.5$）以抑制目标函数的迭代震荡。同时，针对拉格朗日乘子，将二分法搜索的初始上界与收敛容差分别设定为 10^9 与 10^{-3}，以保障体积约束在每一步迭代中均能被高精度满足。基于 OC 算法的优化器实例化及其高级参数的设置过程如代码 6.6 所示。

代码 6.6 基于 OC 的优化器实例化

```
from soptx.optimization.oc_optimizer import OCOptimizer
# 实例化优化器: OC
optimizer = OCOptimizer(
    objective=compliance_objective,
    constraint=volume_constraint,
    regularization_filter=filter_regularization,
    options={
        'max_iterations': 200,            # 最大迭代次数
        'change_tolerance': 1e-2,         # 收敛容差
    }
)
# 配置高级迭代参数
optimizer.options.set_advanced_options(
    move_limit=0.2,        # 移动限制
    damping_coef=0.5,      # 阻尼系数
    initial_lambda=1e9,    # lambda 搜索上界
    bisection_tol=1e-3     # 二分法容差
)
```

在正式启动优化迭代前，需定义设计变量的初始状态。得益于 SOPTX 平台“设计与分析解耦”的核心架构与原生多分辨率支持，设计变量与物理场状态变量可独立定义于不同分辨率甚至不同类型的网格上。因此，在调用插值方案的 `setup_density_distribution` 接口时，需显式传入 `design_variable_mesh`（设计变量网格）与 `displacement_mesh`（位移网格，即底层的物理分析网格）。本算例为展示基础的单分辨率形式，将两者绑定为同一 `mesh` 实例，同时将全域单元的初始密度统一设定为 0.4，以严格满足初始体积分数约束。

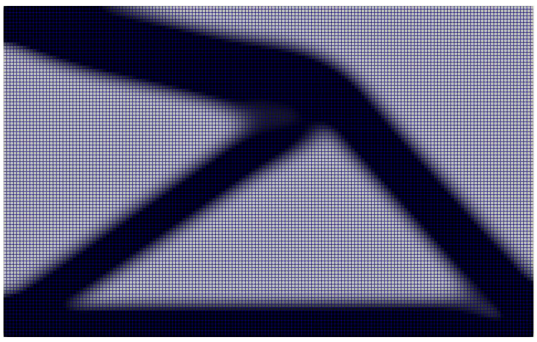
完成状态初始化后，将设计变量与密度场传入优化器的 `optimize` 接口，即可正式驱动材料分布的演化与迭代闭环。在该闭环中，SOPTX 平台将自动串联前述

的有限元分析、灵敏度求解、正则化过滤与算法更新等所有解耦组件。该过程最终将输出优化后的物理密度场 rho_opt 以及收敛历史数据 history（包含各迭代步的目标函数值与约束满足情况）。上述从状态初始化到迭代执行的完整调用逻辑如代码 6.7 所示。

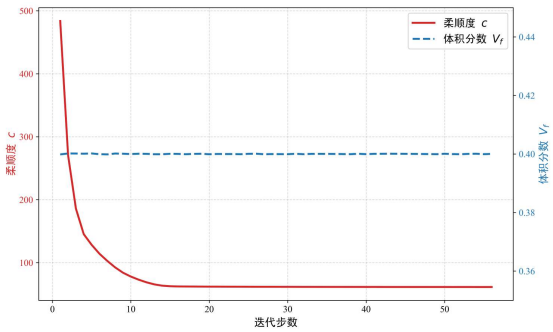
代码 6.7 初始密度场定义与优化过程执行

```python
if __name__ == "__main__":
    # 1. 初始化设计变量与密度场
    d, rho = interpolation_scheme.setup_density_distribution(
        design_variable_mesh=mesh,          # 设计变量网格
        displacement_mesh=mesh,             # 位移网格
        relative_density=0.4,               # 初始密度场
    )
    # 2. 执行优化循环
    rho_opt, history = optimizer.optimize(
        design_variable=d,
        density_distribution=rho
    )
```

执行上述脚本后，即可完成整个拓扑优化流程的离散求解，所得数值结果如图 6.5 所示。其中，图 6.5(a) 为最终的材料密度分布，呈现出清晰的黑白二值化拓扑特征；图 6.5(b) 记录了完整的收敛历程，算法在迭代 56 次后达到收敛判定准则，最终目标柔顺度稳定在 61.4422。分析收敛曲线可见，柔顺度历程平滑且具有良好的单调收敛特性，同时体积分数始终严格锁定在目标值 0.4 附近，未出现数值波动或越界现象。



(a) 最终优化构型



(b) 柔顺度与体积分数收敛曲线

图 6.5 基于 SOPTX 平台的二维悬臂梁优化数值结果

6.5.3 核心类接口与自定义扩展说明

SOPTX 平台提供了开箱即用的标准求解流程，并通过引入“策略模式”与“工厂模式”，构建了高内聚、低耦合的模块化接口体系，实现了算法组件的“即插即用”。本节将从物理建模与变量表征、数值分析策略、优化模型与算法配置以及统一执行环境配置四个维度，深入剖析平台的核心接口设计与自定义扩展机制。借助这些接口，用户仅需调整少量配置项，即可灵活完成离散模型、灵敏度计算路径、优化策略以及底层执行环境的切换与扩展。

1. 物理建模与设计变量表征

作为拓扑优化流程的起点，SOPTX 提供了灵活的物理模型定义与设计变量表征接口，支持对物理问题与优化对象的解耦描述。

物理模型的参数化定义与自定义扩展 SOPTX 平台将偏微分方程的数学定义（包括几何区域、边界条件、载荷工况等）封装为独立且可重用的模型对象。为了兼顾使用的便捷性与扩展的灵活性，平台构建了多层次的模型管理体系：

- **参数化基准模型库**：预置了线弹性刚度设计（如 MBB 梁、悬臂梁）及柔顺机构设计（如位移反向器）等经典问题。用户既可快速开展基准测试，也可灵活调整设计域尺寸与边界载荷以适应特定实验需求。
- **通用偏微分方程抽象接口**：针对非标准工程问题，平台定义了 `PDEBase` 抽象基类。用户仅需继承该类并重写核心变分算子、边界标记及载荷分布等方法，即可将全新物理场无缝接入后续的有限元分析与优化流程中。

代码 6.8 展示了平台调用内置参数化模型的便捷性，以及通过继承基类自定义新物理模型的核心扩展机制。

代码 6.8 物理模型的参数化调用与自定义扩展示例

```python
# --- A. 使用内置模型库（参数化配置） ---
from soptx.model.mbb_beam import HalfMBBBeamRight2d
from soptx.model.displacement_inverter import
    DisplacementInverterUpper2d
# 1. 默认参数模式（适用于快速验证）
pde_default = HalfMBBBeamRight2d()
# 2. 自定义参数模式（灵活调整几何与载荷）
pde_param = DisplacementInverterUpper2d(
    domain=[0, 40, 0, 20], f_in=1.0, f_out=-1.0, k_in=0.1, k_out=0.1
) # 其他基础参数省略...

```

```python
# --- B. 自定义新模型（标准接口扩展示例） ---
from soptx.model.pde_base import PDEBase
class MyCustomBeam(PDEBase):
    """用户仅需继承 PDEBase 并在类内重写相应的边界与载荷方法，
    即可将全新物理场无缝接入平台，具体实现此处省略。"""
    pass
```

多分辨率设计变量表征 SOPTX 原生支持设计域与分析域的解耦（即 MTOP）。在插值方案接口 MaterialInterpolationScheme 中，分别传入设计变量网格和位移网格，并指定单元细化倍率 sub_density_element，平台即可自动构建两者之间的投影算子。这一机制允许用户在保持有限元求解规模（粗网格）不变以控制计算成本的同时，利用高分辨率的设计网格（细网格）清晰捕捉复杂的拓扑边界。如代码 6.9 所示，通过简单调整网格对象并指定子单元映射数量，即可实现多分辨率模式的无缝切换。

代码 6.9 MTOP 的配置与映射接口

```python
# 1. 定义高分辨率设计网格
# 对基准分析网格 (160x100) 进行 2x2 加密，以获得更清晰的拓扑边界
mesh_design = pde.init_mesh(nx=160*2, ny=100*2)

# 2. 建立多分辨率映射关系
# 平台底层自动构建从细网格到粗网格的投影映射算子
d_fine, rho_coarse = interpolation_scheme.setup_density_distribution(
    design_variable_mesh=mesh_design,  # 传入高分辨率细网格
    displacement_mesh=mesh_analysis,   # 保持原有的低分辨率粗网格
    sub_density_element=4,             # 声明比例：1个分析单元包含4个设计单元
    relative_density=0.5               # 初始密度场
)

# 接口输出契约：
# d_fine     : 细网格上的设计变量张量（驱动优化器更新）
# rho_coarse : 粗网格上的物理密度张量（驱动有限元组装）
```

2. 有限元分析策略的灵活配置

为了兼顾计算精度、求解效率与理论扩展性，SOPTX 将有限元分析过程中的几何离散、代数求解与变分原理进行了全方位的解耦与封装。

网格拓扑与有限元阶次的切换 在有限元分析器的设计中，SOPTX 采用了“构建即定义”的不可变设计原则。由于空间离散阶次的改变将导致全局自由度编号与稀疏矩阵结构的彻底重构，平台避免了易引发状态冲突的动态属性修改，而是通过统一的初始化接口支持不同网格拓扑与离散阶次的快速配置。如代码 6.10 所示，用户仅需在实例化 LagrangeFEMAnalyzer 时传入不同的配置参数，即可将默认的线性四边形单元无缝切换为二次三角形单元。这种设计不仅严格确保了底层离散系统的数值一致性，也为本文第三章中关于高阶有限元特性的对比研究提供了极其便捷的代码验证途径。

代码 6.10 网格拓扑与有限元空间阶次的切换配置

```
# 1. 切换离散策略：使用三角形网格
pde.init_mesh.set('uniform_tri')
mesh_tri = pde.init_mesh(nx=160, ny=100)

# 2. 重新实例化分析器：由 Q1 平滑切换至 P2 单元
lagrange_analyzer = LagrangeFEMAnalyzer(
    disp_mesh=mesh_tri,       # 传入新的三角形网格
    # ... 保持 pde, material, interpolation_scheme 等物理定义不变 ...
    space_degree=2,           # [核心变化] 空间离散提升为：二阶单元
    integration_order=4,      # [核心变化] 匹配 4 阶高斯积分
    # ... 保持线性求解器与组装策略配置不变 ...
)
```

线性求解器与组装策略的配置 面对从数千到百万级不同自由度规模的优化问题，单一的计算策略往往难以同时兼顾效率、内存开销与数值稳定性，为此，SOPTX 将矩阵组装与线性方程组求解封装为可配置的策略选项。在矩阵组装层面，平台提供了四种机制以适应不同的离散需求：用户可选择通用性最强的标准组装（standard）或优化内存的 Voigt 组装（voigt）；针对拓扑优化中全局刚度矩阵需频繁重组的计算瓶颈，平台重点引入了基于数值求积的快速组装（fast）与基于解析预计算的符号组装（symbolic）。两者均通过分离单元无关项以大幅降低重复计算开销，区别在于前者基于数值积分预计算参考张量，而后者基于解析符号积分。在方程求解层面，平台支持直接法与迭代法的无缝切换：对于中小规模问题，推荐使用数值鲁棒性较高的直接求解器（如 scipy 或 mumps）；而针对百万级自由度的大规模问题，则可切换至预处理共轭梯度法（cg）或广义最小残差法（gmres），以充分利用并行计算优势并控制内存开销。如代码 6.11 所示，用户仅需修改配置参数，即可实现计算策略的灵活组合。

代码 6.11 线性求解器与组装策略的切换配置

```
# 1. 生成大规模高分辨率网格（百万级自由度测试场景）
```

```
mesh_refine = pde.init_mesh(nx=1000, ny=500)

# 2. 针对大规模问题的性能优化配置
lagrange_analyzer_large = LagrangeFEMAnalyzer(
    disp_mesh=mesh_refine,
    # ... 保持 pde, material, interpolation_scheme 等基础依赖不变 ...
    assembly_method='fast',          # 矩阵快速组装机制
    solve_method='cg',               # 预处理共轭梯度法
)
```

分析变分原理的切换 针对应力约束或近不可压缩材料等对局部应力场逼近精度要求较高的场景，平台支持切换底层求解的变分原理。如本文第五章理论所述，用户可将基于经典最小势能原理的拉格朗日位移型有限元分析器，平滑切换为基于 Hellinger-Reissner 变分原理的胡张混合有限元分析器（`HuZhangMFEMAnalyzer`）。为了从底层保证混合离散格式的数值稳定性，SOPTX 内部封装了应力空间与位移空间的最佳阶次匹配规则。如代码 6.12 所示，当用户指定空间阶次 `space_degree=3` 时，平台将自动构建 P_3 次应力空间与 P_2 次位移空间的稳定离散组合（即 P_k/P_{k-1} 格式），并可通过 `use_relaxation` 参数一键开启顶点应力连续性松弛技术。这种设计使用户在无需手动干预复杂离散细节的情况下，即可保证混合元分析的收敛性与精度。

代码 6.12 分析变分原理的切换：从位移元到混合元

```
# 引入胡张混合有限元分析器
from soptx.analysis.huzhang_mfem_analyzer import HuZhangMFEMAnalyzer

# 重新实例化分析器：由位移元切换为应力-位移混合元 (sigma, u)
mfem_analyzer = HuZhangMFEMAnalyzer(
    # ... 保持 mesh, pde, material 等基础依赖不变 ...
    space_degree=3,          # 指定空间阶次 k=3 (平台将自动匹配 P3/P2
        稳定组合)
    integration_order=5,     # 匹配所需的高阶数值积分
    use_relaxation=True,     # 启用混合元特有的角点松弛技术
    # ... 求解器与组装策略沿用已有配置 ...
)
```

3. 优化问题构建与算法策略配置

在确定了物理场与离散模型后，SOPTX 将优化问题的数学抽象（目标函数、约束条件）、灵敏度计算路径、正则化算子与优化器统一封装为可独立组装的配置项。通过这种模块化的解耦设计，平台不仅能够支持复杂结构设计场景下优化模型的快

速构建，还能够实现不同灵敏度计算方式、正则化策略与优化算法之间的灵活组合与切换，从而使研究人员能够将更多精力集中于新型优化算法与数学规划模型的理论推演，而无需过多介入底层计算代码的重构与维护。

目标函数与约束条件的模块化扩展 在拓扑优化中，不同的工程应用对应着迥异的数学规划模型。SOPTX 将目标函数与约束条件抽象为独立的类对象，不仅封装了前向物理量的计算逻辑，更在底层统一提供了基于伴随法或自动微分的梯度契约接口。除了第 6.5.2 节中展示的经典柔顺度最小化与体积约束外，当优化问题进一步扩展至具有严格局部应力约束的复杂设计场景时，用户只需引入并替换相应的功能模块即可。特别地，如本文第五章所述，面对大规模局部应力约束导致的计算瓶颈，平台原生支持了基于 ALM 的应力约束求解框架。如代码 6.13 所示，用户可以极简地构建一个以体积最小化为目标，并包含严格局部应力约束的复杂优化模型。在这一架构下，目标对象与约束对象被共同注入统一的 `AugmentedLagrangianObjective` 顶层容器中，由底层自动完成拉格朗日乘子与惩罚因子的演化更新。通过将这些算子绑定至胡张混合有限元分析器，平台能够从底层保障局部应力场的逼近精度与优化迭代的收敛性。

代码 6.13 目标函数与约束条件的扩展：基于 ALM 的局部应力约束设计

```python
# 引入体积目标、局部应力约束以及增广拉格朗日目标组件
from soptx.optimization.volume_objective import VolumeObjective
from soptx.optimization.apparent_stress_constraint import
    ApparentStressConstraint
from soptx.optimization.augmented_lagrangian_objective import
    AugmentedLagrangianObjective

# 1.目标函数切换：将优化目标改为最小化结构体积
objective = VolumeObjective(analyzer=mfem_analyzer)

# 2.约束条件扩展：引入表观应力约束
constraint = ApparentStressConstraint(
    analyzer=mfem_analyzer,
    stress_limit=100.0,     # 设定许用应力上限为 100
)

# 3.模型重构：将体积目标与应力约束封装为统一的增广拉格朗日目标函数
augmented_lagrangian_objective = AugmentedLagrangianObjective(
    volume_objective=objective,
    stress_constraint=constraint,
)
```

灵敏度计算路径的切换 SOPTX 支持手动灵敏度与自动微分两种计算路径的统一配置。默认情况下，平台采用基于手动推导的灵敏度实现；当需要启用自动微分时，用户需切换至支持自动微分的计算后端，如 PyTorch 或 JAX，相关后端配置方式将在后文“多后端与设备的统一执行配置”中说明。在具体实现中，目标函数与约束函数的灵敏度计算方式由参数 `diff_mode` 控制，其中 `'manual'` 表示采用手动实现的灵敏度计算，`'auto'` 表示采用自动微分。该参数可分别作用于不同的目标与约束模块，从而支持在同一优化问题中分别为不同目标与约束模块指定灵敏度计算路径。代码 6.14 给出了一个典型示例：对结构柔顺度目标函数启用自动微分，而对体积约束仍采用手动灵敏度实现。

代码 6.14 自动微分灵敏度路径的配置示例

```
# 对柔顺度目标函数启用自动微分
compliance_auto = ComplianceObjective(
    analyzer=lagrange_fem_analyzer,
    diff_mode='auto'
)
# 对体积约束仍采用手动灵敏度实现
constraint_manual = VolumeConstraint(
    analyzer=lagrange_fem_analyzer,
    volume_fraction=0.5,
    diff_mode='manual'
)
```

正则化方法的选择 为了有效抑制拓扑优化中常见的棋盘格现象与网格依赖性问题，SOPTX 内置了多种正则化算子。用户通过配置 `Filter` 类的参数，即可实现不同滤波机制的切换。除了灵敏度过滤（`'sensitivity'`）与密度过滤（`'density'`），平台还支持非线性投影正则化方法（`'projection'`），以减弱单纯密度过滤在结构边界附近引入的灰度带。具体而言，平台内置了双曲正切型（`'tanh'`）与指数型（`'exponential'`）两种光滑连续投影算子。如代码 6.15 所示，用户可灵活设置决定灰度分界点的投影阈值 η（`eta`）以及控制逼近阶跃特征的陡峭度参数 β（`beta`）。在实际优化中，若初始阶段直接采用过大的 β 值，容易使目标函数陷入局部最优，或导致梯度消失等数值困难。因此，平台支持通过配置 β 的连续化演化步长，使平滑密度场在保持迭代稳定性的前提下，逐步向黑白分明的 0-1 物理密度场收敛。

代码 6.15 正则化策略与投影参数的配置

```
from soptx.regularization.filtering import Filter
# 1. 独立配置非线性投影的参数字典
proj_config = {
    'projection_type': 'tanh', # 双曲正切型投影
```

```
    'beta': 1.0,                # 初始陡峭度
    'beta_max': 512.0,          # 最大陡峭度（防止数值奇异）
    'continuation_iter': 50,    # 连续化步长：每50步更新一次beta
    'eta': 0.5                  # 投影阈值
}
# 2. 实例化滤波器并注入配置
filter_regularization = Filter(
    filter_type='projection',  # 切换正则化策略为非线性投影
    projection_config=proj_config
    # ... 保持设计变量网格，过滤半径不变 ...
)
```

优化算法的切换　针对单约束凸问题（如柔顺度最小化），推荐使用计算高效的 OC 法；面对复杂的多约束或非线性问题，则可更改类名平滑切换至 MMA 算法。特别地，SOPTX 的 MMA 优化器原生支持经典带约束与 ALM 框架下的无约束两种调用范式。对于传统优化列式，物理约束对象被显式传入标准 MMA 优化器；而对于含大规模局部应力约束的设计，平台提供了专用的 ALM-MMA 联合框架。在该框架下，由于约束已作为惩罚项吸收入增广拉格朗日目标函数，外层乘子更新与内层 MMA 无约束寻优由底层自动协同完成。如代码 6.16 所示，用户仅需配置内外层双重迭代参数，优化器即可自动驱动设计变量演化。这两种模式的便捷切换，极大地提升了平台对前沿复杂优化理论的兼容能力。

代码 6.16 优化算法的切换：面向不同规划模型的 MMA 实例化配置

```
# --- 模式 A: 经典带约束 MMA 范式 ---
from soptx.optimization.mma_optimizer import MMAOptimizer, MMAOptions
# 1. 配置标准 MMA 算法参数
options_standard = MMAOptions(
    max_iterations=500, change_tolerance=1e-2,
    move_limit=0.2, asymp_init=0.5, asymp_incr=1.2, asymp_decr=0.7
)
# 2. 实例化标准 MMA 优化器（显式传入物理约束）
optimizer_standard = MMAOptimizer(
    objective=compliance_objective, constraint=volume_constraint,
    filters=filter_regularization, options=options_standard
)

# --- 模式 B: ALM 框架下的无约束 MMA 范式 ---
from soptx.optimization.al_mma_optimizer import ALMMMAOptimizer,
    ALMMMAOptions
# 1. 配置 ALM-MMA 联合框架参数（包含内外双层循环控制）
options_alm = ALMMMAOptions(
```

```
    max_al_iterations=150, mma_iters_per_al=5,
    change_tolerance=0.002, stress_tolerance=0.003,
    mu_0=50.0, mu_max=10000.0, alpha=1.1,
    lambda_0_init_val=0.0, move_limit=0.15,
    asymp_init=0.2, asymp_incr=1.2, asymp_decr=0.7, osc=0.2
)
# 2. 实例化 ALM-MMA 优化器（无约束降维模式）
optimizer_alm = ALMMMAOptimizer(
    objective=augmented_lagrangian_objective,
    filters=filter_regularization, options=options_alm
)
```

4. 多后端与设备的统一执行配置

SOPTX 依托 FEALPy 的张量后端管理器，支持计算后端与执行设备的统一配置。用户可通过统一接口在 NumPy、PyTorch、JAX 等后端之间切换，并在相应后端支持下进一步指定 CPU 或 GPU 作为执行设备，而无需修改高层建模与优化流程。

多后端切换 SOPTX 默认采用 NumPy 后端执行计算。借助统一的 `set_backend` 接口，用户可在不改变高层建模与求解流程的前提下，将底层数值执行环境切换为 PyTorch 或 JAX 等其他后端，从而实现同一拓扑优化程序在不同张量计算框架下的复用。常见的后端切换方式如代码 6.17 所示。

代码 6.17 多后端切换的统一配置接口

```
bm.set_backend('numpy')     # 切换到 NumPy 后端
bm.set_backend('pytorch')   # 切换到 PyTorch 后端
bm.set_backend('jax')       # 切换到 JAX 后端
```

执行设备的指定 在所选后端支持设备管理的条件下，用户还可进一步指定张量计算的执行设备。常用方式包括：通过后端管理器统一设置默认设备，使后续新创建的张量默认分配到指定设备；或在网格初始化阶段显式指定设备，使与该网格关联的张量对象及后续计算优先在相应设备上创建与执行。前者适用于整体计算流程统一迁移的场景，后者则具有更高的灵活性，允许用户在同一程序中根据需要分别构造运行于 CPU 或 GPU 上的计算对象，从而实现混合设备执行。相应的设备配置方式如代码 6.18 所示。

代码 6.18 异构设备切换的统一配置接口

```
# 方式 1: 统一设置默认设备
```

```
bm.set_default_device('cuda')

# 方式 2: 在网格创建时显式指定设备
mesh_cuda = pde.init_mesh(nx=160, ny=100, device='cuda')
```

6.6 数值算例与平台能力验证

本节旨在通过典型数值算例，对所开发的拓扑优化软件平台 SOPTX 进行系统验证与综合性能评估。在统一的计算基准下，本节将依次探讨高效矩阵组装策略的性能收益、自动微分灵敏度分析的数值一致性，以及多后端（NumPy/PyTorch/JAX）与异构硬件（CPU/GPU）切换下的跨平台加速能力。通过严格的对照实验与底层耗时拆解，全面展示 SOPTX 平台在应对大规模复杂结构拓扑优化问题时的高效性、可靠性与扩展性。

6.6.1 算例设置与统一框架

为系统评估 SOPTX 平台在计算效率、数值一致性及跨平台执行能力等方面的性能，本节选取经典的三维悬臂梁作为基准测试算例。该算例的物理模型定义、边界条件设置及设计域几何描述与第 ?? 节保持一致。为保证本章数值算例在统一且可复现的基准下进行，除特别说明外，均沿用第 ?? 节中关于小变形线弹性假设、各向同性材料、SIMP 材料插值方案（惩罚因子 $p = 3$）以及 mm/N/MPa 单位体系等通用设定。本章在此基础上，为了严格控制性能对比中的变量，主要补充如下统一约定：

网格与离散规模 ：基准网格设定为均匀六面体网格，剖分尺寸为 $60 \times 20 \times 4$（对应单元总数 4,800，系统全局自由度总数 19,215），有限元分析中的位移场统一采用 $k = 1$ 次拉格朗日插值。

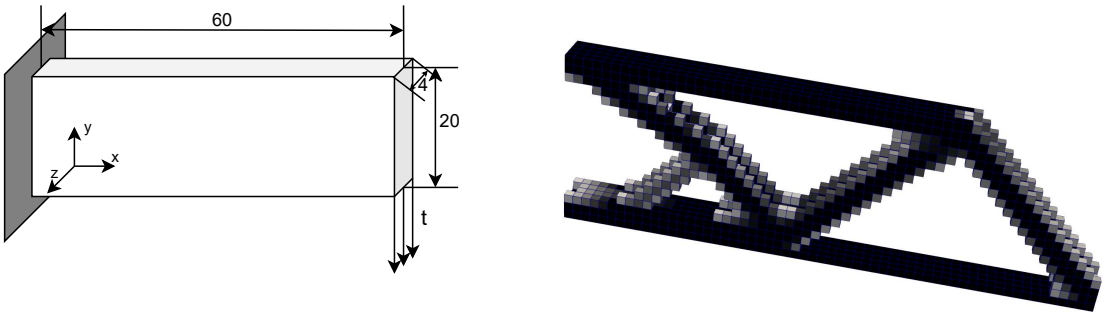
张量计算后端 ：为确立公平且统一的性能比较基准，除第 6.6.3 节（涉及自动微分驱动的灵敏度计算）及第 6.6.4 节（涉及多后端对比及 GPU 异构加速）需显式指定特定后端外，本章其余算例的默认底层张量计算后端均严格设定为 NumPy。

软硬件测试环境 ：所有基于 CPU 的单核或多线程基准测试，均统一运行于搭载 AMD Ryzen 9 9950X 处理器及 64GB 内存的 Ubuntu 24.04 操作系统平台；涉及异构加速的测试，则统一在设备端运行于 NVIDIA GeForce RTX 5070 Ti 计算卡，以评估大规模张量运算的端到端加速比。

6.6.2 高效矩阵组装的能力验证与性能评估

为验证第 6.4.1 节提出的高效矩阵组装技术在实际拓扑优化流程中的性能收益，本节直接采用第 ?? 节中定义的典型三维悬臂梁作为性能评估的基准算例。在严格遵循第 6.6.1 节确立的统一基准框架，并保持网格离散、材料模型及优化参数设置完全一致的前提下，本文重点对比三种不同的刚度矩阵组装实现路径：常规矩阵组装、基于数值求积的快速组装以及基于解析预计算的快速组装。

图 6.6 展示了该基准算例在当前离散下的最终拓扑构型。需要强调的是，由于不同的组装策略仅优化了计算过程而未改变离散模型的数学本质，三者得到的优化目标值与拓扑结果在数值误差范围内保持严格一致（结果如图 6.6(b) 所示，且与第 ?? 节结果吻合）。基于此数值等价性，本节后续的讨论将不再重复物理结果的分析，而完全聚焦于不同策略带来的计算耗时差异。



(a) 物理模型与边界条件　　(b) 最终拓扑构型

图 6.6 三维悬臂梁的物理模型与最终拓扑构型

本算例总迭代次数为 54。为区分一次性预处理开销与稳态迭代开销，本文将计时指标划分为“首次迭代”与“后续迭代平均”两类：首次迭代通常包含稀疏矩阵结构预分配、参考单元积分/映射缓存构建、装配索引生成等一次性操作；而后续迭代平均耗时更能反映拓扑优化循环的稳态计算成本。表 6.1 对比了三种矩阵组装策略在该算例中的耗时结果，包括总耗时、首次迭代耗时与首次矩阵组装耗时，以及后续迭代阶段的平均迭代耗时与平均矩阵组装耗时。其中解析预计算策略的“首次矩阵组装耗时”包含参考算子生成与缓存开销。

从表 6.1 可见，基于数值求积的快速组装显著降低了拓扑优化迭代中的装配开销，并带来端到端耗时下降：总耗时由 92.109 s 降至 57.538 s，整体加速约 1.6 倍。更关键的是，后续迭代阶段的平均矩阵组装耗时由 0.787 s 降至 0.182 s，装配环节加速约 4.32 倍，表明该策略精准且有效地缓解了“高频重装配”这一核心计算开销。

相比之下，基于解析预计算的快速组装在后续稳态阶段的平均装配耗时与前述策略基本持平（0.177 s 对比 0.182 s，差异在浮点计时误差范围内），但首次迭代与首次装配耗时显著增大（57.772 s 与 28.909 s），导致单次完整优化的总耗时上升至

表 6.1 不同矩阵组装技术在三维悬臂梁优化问题中的性能对比（单位 s）

矩阵组装技术	总耗时	首次迭代耗时	首次矩阵组装耗时	后续平均迭代耗时	后续平均矩阵组装耗时
常规矩阵组装	92.109	3.218	1.053	1.677	0.787
基于数值求积的快速组装	57.538	1.772	0.281	1.052	0.182
基于解析预计算的快速组装	113.720	57.772	28.909	1.053	0.177

113.720 s。该差异主要来源于解析预计算策略在首次调用阶段引入的一次性预处理开销：其核心预积分张量 $\hat{S}_{ij}^{kl}$ 仅依赖于单元类型与插值阶次，理论上可在初始化阶段预先计算并缓存，甚至通过离线预计算与持久化缓存实现跨多次求解复用，从而降低首迭代开销，并在重复求解或高阶离散场景下改善端到端效率。与此同时，在仿射映射且被积函数为多项式的情形下，解析/闭式积分可在参考单元层面给出积分的精确值（仅保留浮点舍入误差），从而减少由求积阶次设置不当引入的误差来源，并降低结果对积分规则选择的敏感性。

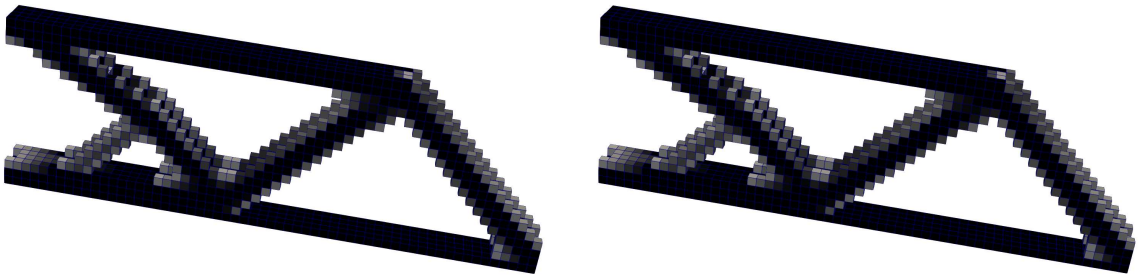
6.6.3 自动微分灵敏度的能力验证与性能评估

为验证第 6.4.2 节所述自动微分灵敏度分析技术在 SOPTX 平台中的工程可用性，本节从正确性与性能代价两方面，对自动微分生成的梯度与手动（解析/伴随）灵敏度实现路径进行对照评估。引入自动微分机制的核心初衷，在于显著降低复杂物理约束及正则化链路下的导数推导与工程实现门槛，并有效规避人为推导误差；而手动解析路径则在本实验中充当数值一致性的绝对校验基准。

实验设置严格遵循第 6.6.1 节确立的统一基准框架。鉴于自动微分机制高度依赖于底层张量计算图的动态追踪特性，本节对照实验统一指定 PyTorch 作为底层计算后端。为精确隔离不同微分方式带来的性能差异，实验设计了严密的控制变量方案：仅将优化目标（结构柔顺度）的梯度获取途径作为唯一变量，分别考察手动解析推导与自动微分端到端求导的差异；同时，两组对照实验中的体积约束梯度均固定采用手动计算。除上述设定差异外，所有的有限元离散流程、网格规模与优化算法参数均保持绝对一致。

图 6.7 给出了两种灵敏度计算方式下三维悬臂梁算例的最终拓扑构型对比。由图可见，无论是基于手动推导还是自动微分获取灵敏度，优化过程均能稳健收敛至拓扑清晰且主承载路径高度一致的构型。两者的杆件空间布局与整体传力通道完全吻合，极微小的底层数值扰动所带来的差异在图像分辨率级别已不可区分。

进一步地，图 6.8 详细给出了结构柔顺度 c 随迭代步数的收敛历程对比。由图可见，代表两种灵敏度计算路径的收敛曲线在全迭代周期内完全重合，且均在第 54



(a) 手动微分　　(b) 自动微分

图 6.7 不同灵敏度计算方式下三维悬臂梁的最终拓扑构型

步精准触发收敛终止准则。由于两组对照实验中的体积约束梯度始终固定采用手动微分，上述高度一致的宏观结果有力地证明了：在本算例与当前平台实现下，自动微分对目标函数（柔顺度）梯度的计算，在底层数值精度范围内完美复现了手动微分的数值结果，从而确保了全局优化轨迹与收敛行为的严格一致。

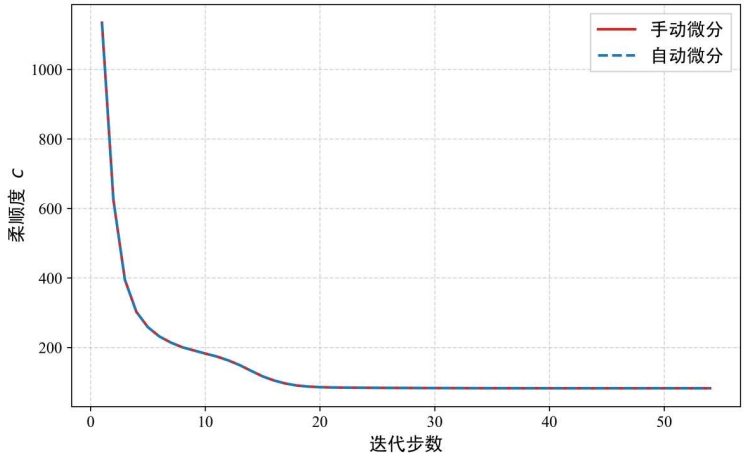


图 6.8 不同灵敏度计算方式下三维悬臂梁的柔顺度收敛历程对比

在确保数值正确性的前提下，表 6.2 进一步对比了两种灵敏度获取路径在三维悬臂梁优化问题中的计算开销。为精准剥离不同计算阶段的性能特征，本文同样将总耗时拆分为“首次迭代耗时”与“后续平均迭代耗时”，以严格区分自动微分引擎引入的一次性初始化成本与稳态计算阶段的边际开销。

表 6.2 不同灵敏度计算方式在三维悬臂梁优化问题中的耗时对比（单位 s）

灵敏度计算方法	迭代次数	总耗时	首次迭代耗时	后续平均迭代耗时
手动微分	54	36.627	1.264	0.667
自动微分	54	36.873	1.460	0.668

由表 6.2 可见，在相同算例设置与相同迭代次数（均为 54 次）下，两条路径的总

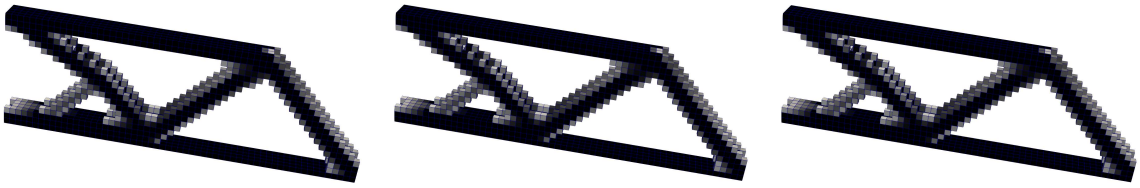
体计算效率几乎一致：自动微分路径的总耗时为 36.873 s，相较手动微分的 36.627 s 仅增加 0.246 s（约 0.67%）；而在进入稳态计算阶段后，两者的平均单步耗时更是几近重合（0.668 s 与 0.667 s，差异约 0.15%）。鉴于手动微分直接执行显式的解析/伴随灵敏度公式，通常具有更短的计算链路与更少的框架性开销，因此常被用作评估灵敏度实现效率的参考基线与近似上界 [？？？]。自动微分能够在该基准上实现性能匹配，充分证实了 SOPTX 平台中基于自动微分的柔顺度灵敏度实现具有极高的底层执行效率，其引入的稳态附加计算开销总体可忽略不计。

进一步拆解耗时差异来源可以发现，自动微分的额外成本主要集中在首次迭代阶段：首次迭代耗时从手动微分的 1.264 s 增至 1.460 s，增加 0.196 s（约 15.5%）。这一现象具有明确的底层算法机制支撑：自动微分引擎在首次调用时，必须动态追踪并构建完整的张量计算图，同时为反向传播预分配中间量缓存，从而产生了必要的一次性初始化开销；而一旦进入稳态迭代，该开销便被迅速摊薄，使得两条路径的总耗时差异保持在极小范围内。综上所述，在本算例与当前平台实现机制下，自动微分能够在不显著牺牲计算效率的前提下，大幅提升复杂物理建模的灵活性与开发效率，从工程应用角度展现出了优异的综合优势与突出的实用价值。

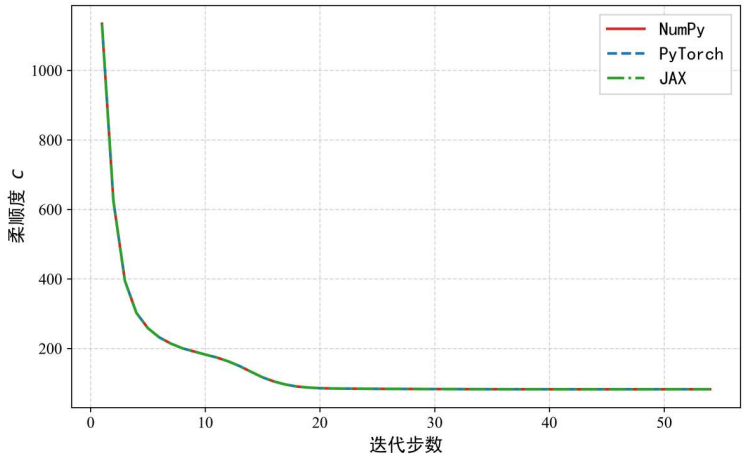
6.6.4 多后端与异构设备切换的能力验证与性能评估

本节旨在验证第 6.4.3 节所述的性能可移植执行机制与后端一致性保障策略在实际计算中的有效性。在统一的计算基准下，本实验将系统考察 SOPTX 平台在多后端（NumPy、PyTorch、JAX）及异构硬件（CPU、GPU）环境下的数值表现。重点评估两方面：一是跨后端计算对拓扑构型与收敛历程的数值一致性保障；二是量化评估 GPU 并行硬件在大规模拓扑优化计算中的加速效果，并深度剖析不同计算阶段与关键算子的性能分布特征，以明确性能提升的具体来源。

首先进行多后端一致性验证。实验严格遵循第 6.6.1 节确立的统一基准框架，在保持物理模型、离散方案及优化参数完全一致的前提下，分别调用 NumPy、PyTorch 与 JAX 三种计算后端执行全流程拓扑优化。图 6.9 综合展示了不同后端下的数值一致性表现。



(a) NumPy (b) PyTorch (c) JAX



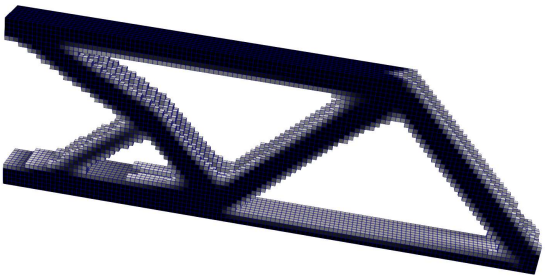
(d) 收敛历程

图 6.9 不同计算后端下的数值一致性验证：(a)-(c) 最终拓扑构型；(d) 柔顺度收敛历程对比

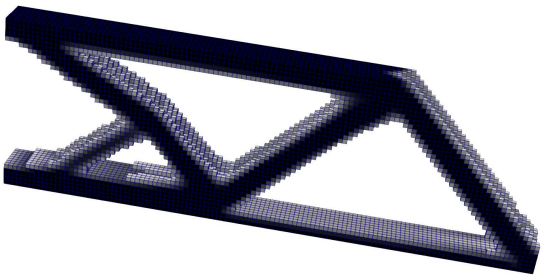
由图 6.9(a)-6.9(c) 可见，三种计算后端均稳健收敛至拓扑清晰且主承载路径严格一致的构型，不同后端在个别边界单元处产生的微小数值扰动，在图像分辨率下已几乎不可分辨。同时，图 6.9(d) 显示三条柔顺度下降曲线在全迭代过程中几乎完全重合，且在相同的终止准则下均于第 54 步准确触发收敛。上述结果有力地证明了 SOPTX 平台的统一张量抽象层能够有效屏蔽底层数值库的实现差异，从算法层面保障了上层优化流程在跨后端切换时的数值稳定性与结果复现性。

为充分考察异构设备切换带来的性能增益，本小节将三维悬臂梁算例的网格规模加密至 $120 \times 40 \times 8$（全局自由度规模 133,947），以提升计算密集度。鉴于自由度规模的显著增加，直接法求解器将面临巨大的内存开销瓶颈，本实验在 CPU 与 GPU 端均统一采用共轭梯度法作为线性方程组求解器。这种设置既保证了对比实验中算法层面的严格一致性，又能有效验证平台在处理大规模稀疏系统时，利用异构硬件加速迭代求解的能力。在进行性能评估之前，首先验证 SOPTX 平台在异构设备切换下的数值可靠性，所得数值结果如图 6.10 所示。

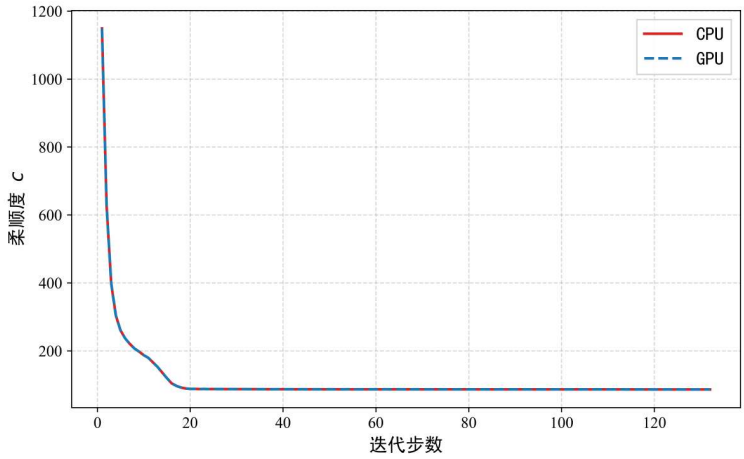
由图 6.10(a)-6.10(b) 可见，CPU 与 GPU 设备下均收敛到了宏观布局完全一致的解，结构的传力路径分布、主要杆件的连通关系均高度吻合，表明底层硬件架构的差异并未改变优化问题的极值解特征。同时，图 6.10(c) 显示代表两种设备的柔顺度下降曲线在全迭代过程中几乎完全重合，且在相同的收敛准则下均准确地在第



(a) CPU



(b) GPU



(c) 收敛历程

图 6.10 不同计算设备下的数值一致性验证：(a)-(b) 最终拓扑构型；(c) 柔顺度收敛历程对比

132 步终止迭代。上述结果有力证明了设备切换未改变优化算法的搜索轨迹。需要指出的是，若深究数值层面的极微小差异（通常在 10^{-14} 量级），其主要归因于并行计算中特有的浮点舍入误差：CPU 通常采用串行累加，而 GPU 在执行大规模矩阵乘法或向量归约时多采用并行的树状累加；由于浮点数加法不满足严格的结合律，运算顺序的改变会导致舍入误差的微小累积。实验结果表明，这种微小的数值扰动完全在工程容许范围内，SOPTX 平台在异构设备间保持了严格的算法级一致性。

在确认数值可靠性的基础上，表 6.3 详细对比了 CPU 与 GPU 在该大规模算例中的计算开销。结果显示，SOPTX 利用 GPU 后端将优化总耗时由 2789.93 s 大幅压缩至 636.74 s，实现了约 4.38 倍的端到端加速。

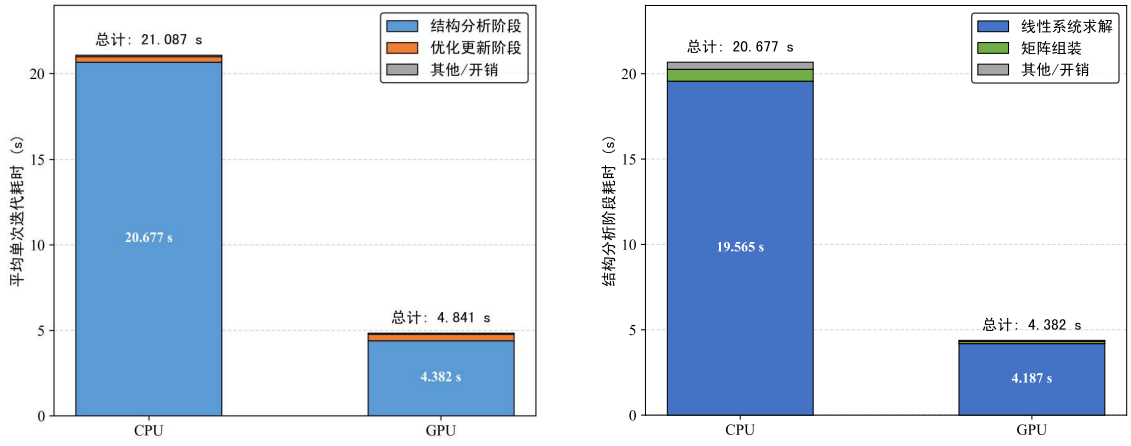
表 6.3 不同设备下三维悬臂梁优化问题的耗时与加速比对比（单位：s）

计算设备	迭代次数	总耗时	端到端加速比	首次迭代耗时	后续平均迭代耗时	稳态加速比
CPU	132	2789.927	—	6.443	21.087	—
GPU	132	636.741	**4.38**×	1.821	4.841	**4.36**×

进一步剖析迭代过程中的动态耗时特征可以发现，GPU 的加速效能在不同阶段均表现优异。在初始化阶段（首次迭代），尽管 GPU 后端涉及底层算子预加载、计算图构建及设备显存预分配等必要的系统开销，但得益于 SOPTX 对张量运算的高效批量化封装，其初始化效率依然得到大幅提升，实现了约 3.54 倍的初步加速。进入稳态计算阶段（后续迭代）后，GPU 的并行计算优势得到进一步释放，平均单步耗时由 21.09 s 降至 4.84 s，稳态加速比提升至 4.36 倍。这表明在当前十万级自由度的问题规模下，GPU 极高的大规模并行吞吐能力使得计算开销在总耗时中占据主导地位，有效弱化了数据传输与内核启动等固定延迟带来的性能损耗，显著提升了迭代效率。

为了进一步解析 GPU 加速效能的底层来源并识别算法的性能瓶颈，本小节对稳态迭代过程中的各项耗时占比进行了多维度的量化剖析。通过对拓扑优化主链路及核心分析算子的逐层拆解，旨在明确异构硬件在大规模并行计算中的优势区间。图 6.11 详尽展示了不同设备下平均单步迭代耗时的构成分布以及结构分析阶段的算子级耗时拆解情况。

由图 6.11(a) 可见，结构分析阶段占据了迭代总开销的绝对主导地位，在 CPU 端占比高达 98.1%，GPU 端亦占 90.5%。在该阶段，GPU 耗时由 20.677 s 锐减至 4.382 s，实现了约 4.72 倍的加速，是驱动端到端性能提升的决定性因素。相比之下，优化更新阶段的耗时在 CPU（0.317 s）与 GPU（0.377 s）上处于同一量级，并未表现出并行加速收益。这主要是由于优化模块主要涉及设计变量的简单代数更新与正则化操作，计算密集度较低，此时数据传输延迟与内核启动开销成为主导因素，限



(a) 稳态迭代阶段平均耗时分布　　(b) 结构分析阶段耗时的算子级拆解

图 6.11 不同设备下三维悬臂梁算例的计算性能剖析

制了加速表现。然而，由于该阶段在总耗时中占比极低，并未对整体加速效果构成性能瓶颈。

针对核心性能瓶颈，即结构分析阶段，图 6.11(b) 的算子级拆解进一步揭示了加速来源。线性系统求解是最大的性能瓶颈，其稳态平均耗时由 CPU 的 19.565 s 降至 GPU 的 4.187 s（加速 4.67 倍），贡献了绝大部分的性能增益。这主要归因于 CG 迭代法的核心算子是稀疏矩阵–向量乘法 (SpMV) 与向量运算，这类计算具有高度的数据并行性；相较于 CPU，GPU 极高的显存带宽与大规模并行核心显著提升了 SpMV 的吞吐量，从而在相同的算法下大幅缩短了求解时间。此外，矩阵组装环节获得了最高的单项加速比（约 5.47 倍，由 0.695 s 降至 0.127 s）。这一结果有力地证明了本文第 6.4.1 节提出的“基于算子缓存与张量并行收缩的快速组装策略”能够充分利用 GPU 的单指令多线程（SIMT）架构，成功将传统耗时的串行装配过程转化为高效的并行张量运算。

综上所述，SOPTX 平台在保障跨后端数值结果严格一致的基础上，通过异构计算策略显著降低了拓扑优化中最耗时的分析环节开销。实验结果充分验证了平台架构在处理大规模工程问题时具备优异的性能可移植性，能够有效释放高性能硬件的并行计算潜力。

6.7 本章小结

本章立足于拓扑优化研究从“理论算法”向“高性能工程实现”的转化需求，设计并实现了基于 FEALPy 的统一张量计算平台 SOPTX。针对传统拓扑优化程序在复杂算子开发、跨平台迁移及大规模装配性能等方面的瓶颈，系统构建了涵盖自动微分、统一张量抽象与异构加速技术的软件体系。通过一系列典型三维基准算例，对

软件的数值正确性、后端一致性及异构计算效能进行了严谨验证。主要结论如下：

1. **统一张量抽象与多后端动态适配机制**：构建了屏蔽底层数值库实现差异的张量算子规约层，实现了“一次开发，多后端执行”的通用化开发范式。通过后端管理器动态调度 NumPy、PyTorch 与 JAX 等主流计算后端，在保障算法高度灵活性的同时，确保了跨计算环境输出结果的严格复现性与数值稳定性。

2. **模块化驱动的多模型与算法解耦扩展能力**：基于高层抽象的模块化架构设计，实现了物理分析、灵敏度计算与优化驱动的深度解耦。平台不仅支持从传统体积–柔顺度模型向多物理场耦合、大规模应力约束等复杂列式的模块化集成与功能拓展，亦能无缝集成 OC、MMA 等多种优化算法，并灵活接入 ALM 等应力约束问题求解框架，显著增强了系统针对复杂工程场景的适配性与可定制潜力。

3. **预积分算子缓存与张量化组装机制**：针对大规模三维问题中矩阵组装的性能瓶颈，提出了将参考单元积分项与几何映射解耦的预计算策略。通过构建算子缓存库并引入张量并行收缩运算，实现了从元素级串行循环向高性能张量化计算的范式转变，显著提升了大规模稀疏线性系统的构建效率。

4. **基于自动微分的高精度灵敏度分析体系**：引入自动微分技术，解决了复杂物理场耦合及非线性材料插值模型下繁琐的手动求导难题。验证表明，SOPTX 生成的自动微分灵敏度与手动推导格式在数值上完全等价，极大地降低了新型优化列式的集成门槛与验证成本。

5. **异构计算加速效能与性能瓶颈深度剖析**：量化评估了 GPU 硬件在大规模并行计算中的性能收益。实验证实，在大规模自由度场景下，GPU 极高的内存带宽与计算吞吐能力能够有效弱化数据传输延迟的影响，在核心分析环节及矩阵组装环节均实现了显著的端到端加速，展现出良好的工程实用价值。

综上所述，SOPTX 平台通过将先进的张量计算技术与现代拓扑优化理论深度融合，有效解决了高性能计算中开发效率与执行效率的固有矛盾。平台在保障数值严谨性的同时，显著增强了大规模三维结构的优化分析效率，为开展多物理场耦合、大规模应力约束等前沿计算力学问题的深入研究提供了坚实且可扩展的计算支撑体系。

第 7 章 总结与展望